

IoT-Based Smart Home Automation System Using ESP32

Final Year Project Report

by

M. Mohsin Saleem	BCS22006
Abdul Rehman Tahir	BCS22016
Rehan Ali	BCS22021
Muhammad Owais	BCS22022
Wasif Ali	BCS22024

Project Advisor

Dr. Salman Naseer

Department of Information Technology

University of the Punjab, Gujranwala Campus, Gujranwala, Pakistan

(2026)

IoT-Based Smart Home Automation System Using ESP32

Executive Summary

This project presents the design and development of a Smart Home Automation System that enables users to monitor and control household devices through a mobile application. The system was conceived to provide convenience, energy efficiency, and real-time responsiveness using affordable hardware and cloud-based services. The Project Proposal outlined the motivation, objectives, and feasibility of the solution, while the Software Requirements Specification (SRS) defined the functional and non-functional requirements, user interactions, and system constraints that guided development.

The Software Design Specification (SDS) translated these requirements into a structured architecture built around object-oriented principles, an ESP32-based hardware abstraction layer, threshold-based automation logic, and a normalized Firebase Realtime Database. The prototype demonstrates the complete workflow—from device control and sensor monitoring to automation rule execution—through a Flutter mobile interface and ESP32 hardware integration. Together, these documents and the working prototype showcase a scalable, maintainable, and user-friendly smart home system aligned with the project's goals of reliable device control, transparent automation, and an intuitive mobile experience.

FYDP Overview

FYDP Title: IoT-Based Smart Home Automation System using ESP32

Table 1: Project Members

Sr. No	Roll Numbers	Name	Signatures
1.	BCS22006	Mohsin Saleem	
2.	BCS22016	Abdul Rehman Tahir	
3.	BCS22021	Rehan Ali	
4.	BCS22022	Muhammad Owais	
5.	BCS22024	Wasif Ali	

Table 2 summarizes the goals, assumptions, risks, and target users of the system.

Table 2: FYDP Goals, Objectives, Success Criteria, Assumptions, Risks, and Target Users

FYDP Goals
▪ Develop a unified IoT-based smart home system using ESP32 microcontroller and Wi-Fi communication. ▪ Provide a secure, user-friendly mobile application using Firebase authentication. ▪ Enable intelligent automation through sensors (temperature, motion, light, water level). ▪ Implement timer-based scheduling and peak-time alerting via buzzer. ▪ Introduce a subscription model for device analytics and scalability.
FYDP Objectives
▪ Provide secure user registration and login through Firebase Authentication. ▪ Implement remote ON/OFF control for ESP32-connected appliances. ▪ Introduce Automatic Mode for environment-based decision-making using sensors. ▪ Ensure seamless communication between Flutter app, Firebase, and ESP32 hardware. ▪ Deliver a scalable and cost-effective smart home solution suitable for everyday households. ▪ Integrate subscription logic to differentiate free and premium device analytics.
FYDP Success Criteria
▪ The system reliably controls ESP32-connected devices through the mobile app. ▪ Automatic Mode responds accurately to sensor inputs (temperature, motion, light, water level). ▪ Timer mode executes scheduled operations correctly. ▪ Peak-time buzzer activates during defined hours. ▪ The mobile application is intuitive, stable, and responsive. ▪ Subscription logic functions correctly for device analytics.

▪ Hardware–software integration remains consistent and error-free.
Assumptions
▪ Appliances such as fans, lights, pumps, and curtains can be connected via ESP32 relays. ▪ A stable internet connection is available for Firebase authentication and data sync. ▪ Users follow safety guidelines when interacting with electrical appliances. ▪ ESP32 devices operate reliably in typical home environments.
Risks & Obstacles
▪ Unstable Wi-Fi may affect remote control and Firebase communication. ▪ Hardware failures (sensors, relays, modules) may disrupt automation. ▪ Integration challenges between Flutter, Firebase, and ESP32 may arise. ▪ Incorrect wiring or incompatible appliances may reduce system reliability. ▪ Subscription logic may require additional testing for scalability.
Organization Address: FCIT, University of the Punjab, Lahore, Pakistan
Target End Users
▪ Homeowners seeking convenience, automation, and centralized control. ▪ Families wanting sensor-based smart home features. ▪ Small offices or residential setups needing affordable automation solutions.
Suggested Project Supervisor:
Approved By:
Date:

Table of Contents

Executive Summary.....	ii
<i>FYDP Overview</i>	<i>iii</i>
12	
<i>Chapter No 1: Project Proposal</i>	12
<i>1.1. Introduction</i>	13
1.1.1 Introduction to IoT.....	13
1.1.2 Characteristics of Constrained Devices.....	13
1.1.3 IETF IoT Protocol Stack	13
1.1.4 Low-Power and Lossy Networks (LLNs)	14
1.1.5 Limitations of LLNs	14
1.1.6 ESP32 Overview	14
<i>1.2. Background</i>	15
<i>1.3. Problem Statement</i>	15
<i>1.4. Stakeholders & Interests.....</i>	15
<i>1.5. Objectives</i>	16
<i>1.6. Scope</i>	16
1.6.1 Inclusion.....	16
1.6.2 Exclusion	16
<i>1.7. Assumptions.....</i>	17
<i>1.8. Risks</i>	17
<i>1.9. Success Criteria</i>	17
<i>1.10. Tools, Libraries & Technologies.....</i>	18
<i>1.11. Work Division.....</i>	20
<i>1.12. Conclusion.....</i>	20
21	
<i>Chapter No 2 Literature Review.....</i>	21
<i>2.1 Literature Survey</i>	22
<i>2.2 Related Work</i>	22
<i>2.3 Gap Analysis</i>	22
<i>2.4 Summary</i>	23

Chapter No 3	24
Software Requirements Specification.....	24
3.1 Requirements Analysis	25
3.2 User classes and characteristics	25
3.3 Requirement Identifying Technique.....	25
3.4 Functional Requirements	26
3.4.1 User Registration (Functional Requirement: FR-1)	26
3.4.2 User Login (Functional Requirement: FR-2)	27
3.4.3 Device Onboarding (Functional Requirement: FR-3)	28
3.4.4 Remote Appliance ON/OFF Control (Functional Requirement: FR-4)	29
3.4.5 Automatic Mode Activation (Functional Requirement: FR-5).....	29
3.4.6 Temperature-Based Fan Control (Functional Requirement: FR-6)	30
3.4.7 Motion-Based Light Automation (Functional Requirement: FR-7).....	31
3.4.8 Light-Level Curtain Automation (Functional Requirement: FR-8)	32
3.4.9 Water-Level Pump Automation (Functional Requirement: FR-9)	33
3.4.10 Peak-Time Buzzer Alert (Functional Requirement: FR-10).....	33
3.4.11 Communication Handling (Functional Requirement: FR-11).....	34
3.4.12 Subscription Management (Functional Requirement: FR-12)	35
3.5 Non-Functional Requirements	36
3.5.1 Reliability	36
3.5.2 Usability	36
3.5.3 Performance.....	36
3.5.4 Security	37
3.6 External Interface Requirements	37
3.6.1 User Interfaces Requirements	37
3.6.2 Software Interfaces	38
3.6.3 Hardware Interfaces.....	38
3.6.4 Communications Interfaces	38
3.7 Use case Analysis	39
3.7.1 Use Case #1: User Registration (UC-1)	39
3.7.2 Use Case #2: Login to the System (UC-2)	40
3.7.3 Use Case #3: Add/Pair a New Device (ESP32) (UC-3)	41
3.7.4 Use Case #4: Control Device ON/OFF (UC-4).....	42

3.7.5	Use Case #5: Activate Automatic Mode (UC-5)	43
3.7.6	Use Case #6: Temperature-Based Fan Automation (UC-6)	44
3.7.7	Use Case #7: Motion-Based Light Automation (UC-7)	45
3.7.8	Use Case #8: Light-Level Curtain Automation (UC-8)	46
3.7.9	Use Case #9: Water-Level Pump Automation (UC-9)	47
3.7.10	Use Case #10: Peak-Time Buzzer Alert (UC-10)	48
3.7.11	Use Case #11: View Device Status (UC-11)	49
3.7.12	Use Case #12: Update Device Settings (UC-12)	50
3.7.13	Use Case #13: Manage Subscription (UC-13)	51
3.7.14	Use Case Diagram	52
3.8	<i>Storyboards</i>	53
3.8.1	Storyboard 1: Registration, Login, and Dashboard Flow	53
3.8.2	Storyboard 2: Device Pairing and Control	53
3.8.3	Storyboard 3: Fan Automation (Temperature Sensor)	53
3.9	<i>Summary</i>	54
	<i>Chapter No 4</i>	55
	<i>Software Design Specification</i>	55
4.1	<i>System Design</i>	56
4.2	<i>Design Considerations</i>	56
4.2.1	Assumptions and Dependencies	56
4.2.2	Limitations	56
4.3	<i>Requirements Traceability Matrix</i>	57
4.4	<i>Design Models</i>	58
4.4.1	Design Class Diagram (DCD)	58
4.4.2	Interaction Diagram (Sequence Diagram)	61
4.4.3	State Transition Diagram	63
4.5	<i>Architectural Design</i>	64
4.5.1	UML Component diagram	65
4.6	<i>Data Design</i>	66
4.6.1	Data Dictionary	67
4.7	<i>User Interface Design</i>	70
4.7.1	Screen Images	70
4.7.2	Screen Objects and Actions	73

4.8	<i>Design Decisions</i>	74
4.8.1	Selection of Object-Oriented Design	74
4.8.2	Choice of Firebase as Backend Platform	74
4.8.3	Database Structure and Normalization Level	74
4.8.4	Placement of Automation Logic on ESP32	74
4.8.5	Communication and Data Exchange Approach	74
4.8.6	Separation of Subscription Management	75
4.8.7	Abstraction of Devices and Sensors	75
4.9	<i>Summary</i>	75
	<i>References</i>	76

List of Tables

Table 1: Project Members	iii
Table 2: FYDP Goals, Objectives, Success Criteria, Assumptions, Risks, and Target Users.	iii
Table 3: Stakeholders & Interests	15
Table 4 Project Success Criteria	17
Table 5: Tools Used	18
Table 6: Flutter Libraries	18
Table 7: ESP32 Libraries	19
Table 8: Technologies used	19
Table 9: Work Division	20
Table 10: User Class and Characteristics.....	25
Table 11 Functional Requirement – User Registration	26
Table 12: Functional Requirement - User login	27
Table 13: Functional Requirement - Device Onboarding.....	28
Table 14: Functional Requirement - Device On/Off	29
Table 15: Functional Requirement - Automatic Mode	29
Table 16: Functional Requirement - Temperature-Based Fan Control	30
Table 17: Functional Requirement - Motion Based Light Automation.....	31
Table 18: Functional Requirement - Light level Curtain Automation.....	32
Table 19: Functional Requirement - Water Pump Automation	33
Table 20: Functional Requirement- Peak-Time Buzzer Alert	33
Table 21: Functional Requirement - Communication Handling.....	34
Table 22: Functional Requirement - Subscription Management	35
Table 23: Use Case Description for "User Registration"	39
Table 24: Use Case Description for "User Login"	40
Table 25: Use Case Description for "Adding/Pairing a new appliance"	41
Table 26: Use Case Description for "Controlling Device "	42
Table 27: Use Case Description for "Automatic Mode Activation "	43
Table 28: Use Case Description for "Temperature-Based Fan Automation "	44
Table 29: Use Case Description for "Motion-Based Light Automation "	45
Table 30: Use Case Description for "Light-Level Curtain Automation "	46
Table 31: Use Case Description for "Water-Level Pump Automation "	47
Table 32: Use Case Description for "Peak-Time Alert "	48

Table 33: Use Case Description for " View Device Status"	49
Table 34: Use Case Description for "Update Device Settings"	50
Table 35: Use Case Description for " Manage Subscription ".....	51
Table 36: Requirements Traceability Matrix.....	57
Table 37: Data Dictionary (OO Approach)	67

Tables of Figures

Figure 1: IoT Layer Architecture	13
Figure 2: ESP32	14
Figure 3: User Registration.....	39
Figure 4: User Login.....	40
Figure 5: Add/Pair a New Device	41
Figure 6: Control Device ON/OFF	42
Figure 7: Automatic Mode Activation	43
Figure 8: Temperature-Based Fan Automation	44
Figure 9: Motion-Based Light Automation	45
Figure 10: Light-Level Curtain Automation.....	46
Figure 11: Water-Level Pump Automation	47
Figure 12: Peak-Time Alert	48
Figure 13: View Device Status	49
Figure 14: Update Device Settings	50
Figure 15: Manage Subscription.....	51
Figure 16: Use Case Diagram.....	52
Figure 17: Class Diagram	60
Figure 18: Interaction Diagram: User turns on Device.....	61
Figure 19: Interaction Diagram - Device Control Interaction	62
Figure 20: Interaction Diagram - Login Process.....	62
Figure 21: Interaction Diagram – Automation Rule Execution.....	63
Figure 22: State Transition Diagram.....	64
Figure 23: Architecture Diagram	65
Figure 24: Component Diagram	66
Figure 25: Splash Screen	71
Figure 26: Register Screen.....	71
Figure 27: Login Screen.....	72
Figure 28: Dashboard Screen.....	72

Chapter No 1: Project Proposal

1.1. Introduction

1.1.1 Introduction to IoT

The Internet of Things (IoT) is a group of interdependent devices (Al-Fuqaha, 2015) that can sense and communicate and perform actions independently. IoT allows the daily appliances, including fans, lights, and TVs, to be supervised and managed with the help of mobile apps or automated logic. The IoT systems comprise, as a rule, sensors, actuators, microcontrollers, communication modules, and cloud services. **Figure 1** shows IoT Layer Architecture.

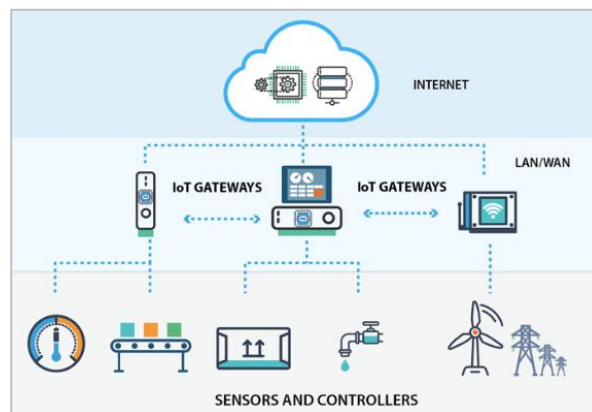


Figure 1: IoT Layer Architecture

1.1.2 Characteristics of Constrained Devices

IoT devices often operate under constraints (Kaur, 2020) such as:

- Limited processing power
- Restricted memory
- Low energy availability
- Unreliable wireless connectivity
- Small packet sizes and low data rates

These constraints influence protocol design, communication patterns, and system architecture.

1.1.3 IETF IoT Protocol Stack

The Internet Engineering Task Force (IETF) defines a lightweight protocol stack (Association, 2025) for IoT, including:

- **MQTT**
- **HTTP/REST**
- **WebSockets**
- **TCP/UDP**

In this project, Firebase Realtime Database is used for cloud communication, while ESP32 handles Wi-Fi connectivity.

1.1.4 Low-Power and Lossy Networks (LLNs)

LLNs consist of devices with limited power, memory, and processing capabilities (Chen, Wang, & Zhang, 2025). They operate over wireless links that may experience interference, packet loss, and limited range. LLNs are widely used in smart homes, sensor networks, and industrial IoT.

1.1.5 Limitations of LLNs

LLNs face several challenges:

- High packet loss
- Limited bandwidth
- Interference
- Short communication range
- Energy constraints

These limitations must be considered when designing reliable smart home systems.

1.1.6 ESP32 Overview

ESP32 is a powerful microcontroller with built-in Wi-Fi and Bluetooth. It supports multiple sensors, real-time communication, and cloud integration. In this project, ESP32 handles:

- Relay-based appliance control
- Temperature, motion, light, and water-level sensing
- Automatic Mode logic
- Timer scheduling
- Peak-time alerting
- Firebase communication

Figure 2 shows ESP32 structure.



Figure 2: ESP32

1.2. Background

Smart home automation has grown rapidly with the advancement of IoT technologies. Traditional systems rely on manual switches or basic timers, lacking intelligent decision-making and centralized control. With microcontrollers like ESP32, cloud platforms such as Firebase, and mobile frameworks like Flutter, it is now possible to build affordable and intelligent automation systems.

1.3. Problem Statement

Households often struggle to manage multiple appliances efficiently. Forgetting to turn off devices, wasting electricity during peak hours, and lacking centralized control are common issues. Existing commercial systems are expensive and require proprietary hardware. There is a need for a smart home system that:

- Controls ESP32-connected appliances remotely
- Automates devices using environmental sensors
- Provides timer-based and peak-time alert functionality
- Offers subscription-based analytics
- Ensures secure access through Firebase
- Provides a unified mobile interface

This project addresses these gaps by developing an integrated, affordable, and intelligent home automation solution.

1.4. Stakeholders & Interests

Understanding who benefits from this smart home system is important because each group interacts with it differently. Homeowners care about convenience and energy savings, renters prefer easy installation, and developers focus on reliability and scalability. Identifying these needs helps shape a system that is practical, usable, and relevant. The main stakeholders are listed in **Table 3**.

Table 3: Stakeholders & Interests

Stakeholder	Interests/Notes	Estimated Market Size
Homeowners	Convenience, automation, energy savings	500,000 - 1,000,000 households
Families	Comfort, ease of use, safety	Included above
Renters	Easy installation, low cost	300,000 – 500,000 households
Developers	IoT + mobile development experience	N/A

Utility Providers	Encourage energy-efficient behavior	N/A
-------------------	-------------------------------------	-----

1.5. Objectives

The objectives of the project are as follows:

- Develop a unified ESP32-based smart home system
- Implement secure Firebase authentication
- Enable sensor-driven Automatic Mode
- Provide timer-based device scheduling
- Implement peak-time buzzer alerts
- Introduce subscription-based analytics
- Build a user-friendly Flutter mobile application
- Ensure real-time communication between app, Firebase, and ESP32

1.6. Scope

The scope of the project is as follows:

1.6.1 Inclusion

The IoT Smart Home Automation System focuses on the design, development, and implementation of a smart home solution that improves energy efficiency, convenience, and sustainability. The system includes:

- Manual ON/OFF control of ESP32-connected appliances
- DHT11 temperature-based fan automation
- PIR motion-based fan/light automation
- Water-level sensor for pump auto-cutoff
- LDR-based curtain automation
- Timer mode
- Peak-time buzzer alerts
- Firebase authentication
- Subscription model (2 devices free, full stats premium)
- Flutter mobile app

1.6.2 Exclusion

The project does not cover:

- Voice control
- Wi-Fi smart TV modules
- Alexa/Google Home integration

- AI-based prediction models
- Industrial automation

1.7. Assumptions

- Appliances such as fans, lights, pumps, and curtains can be connected via ESP32 relays.
- A stable internet connection is available for Firebase authentication and data sync.
- Users follow safety guidelines when interacting with electrical appliances.
- ESP32 devices operate reliably in typical home environments.

1.8. Risks

- Unstable Wi-Fi may affect remote control and Firebase communication.
- Hardware failures (sensors, relays, modules) may disrupt automation.
- Integration challenges between Flutter, Firebase, and ESP32 may arise.
- Incorrect wiring or incompatible appliances may reduce system reliability.
- Subscription logic may require additional testing for scalability.

1.9. Success Criteria

The success criteria of the project is as follows:

Table 4 Project Success Criteria

Sr. No.	Success Criterion
1	Real-time device control (ON/OFF)
2	Smart Light Auto Mode functionality
3	Fan automation using temperature threshold
4	Manual override for all automated devices
5	Accurate sensor data retrieval (temperature, humidity, light)
6	Fast and reliable Firebase Realtime Database synchronization
7	Stable ESP32 communication and hardware response
8	Intuitive and user-friendly mobile UI (Flutter)
9	Speed and responsiveness of device commands
10	Proper error handling and device offline detection
11	Secure access to device data and communication
12	Scalability for adding new device types or automation rules
13	Clean, modular Object-Oriented architecture
14	Cross-platform mobile compatibility (Android/iOS)
15	Fully functional prototype demonstrating end-to-end workflow

16	Deployment readiness with documented setup steps
----	--

1.10. Tools, Libraries & Technologies

This project integrates hardware components, software frameworks, and cloud services to build a unified smart home automation system. The selected tools and technologies ensure reliable communication between the mobile application, Wi-Fi modules, ESP32 hardware, and Firebase backend. **Table 5** summarizes the tools used in the project along with their rationale.

Table 5: Tools Used

	Tools	Version	Rationale
Tools	ESP32 IDE / Arduino IDE	2.1.1	Used to program ESP32 boards, upload code, and manage sensor/relay control.
	Visual Studio Code (Flutter IDE)	Latest	Lightweight IDE with strong Flutter/Dart support for building the mobile app.
	GitHub	Latest	Version control for collaboration, Code tracking and project management.

Table 6 shows the flutter libraries used in the project along with their rationale.

Table 6: Flutter Libraries

	Libraries	Version	Rationale
Libraries (Flutter)	Flutter Core Libraries	Latest	Handles UI, navigation, and device-control screens.
	Firebase Authentication Library	Latest	Provides secure login and user identity management.
	Firebase Realtime Database / Firestore	Latest	Stores device states and syncs data between app and hardware.
	State Management Library (Provider / GetX)	Latest	Ensures smooth app state Handling and real-time updates.

Table 7 shows the ESP32 libraries used in the project along with their rationale.

Table 7: ESP32 Libraries

Libraries (Arduino)	Libraries	Version	Rationale
	ESP32 Wi-Fi Libraries	Latest	Enables Wi-Fi communication for controlling smart devices.
	DHT Sensor Library	Latest	Reads temperature data for Automatic Mode.
	LDR Sensor Library	Latest	Detects light levels for curtain automation.
	PIR Sensor Library	Latest	Detects room occupancy for energy-saving automation.
	Water-Level Sensor Library	Latest	Monitors tank level for pump control.

Table 8 shows the flutter libraries used in the project along with their rationale.

Table 8: Technologies used

Technologies	Technologies	Version	Rationale
	ESP32	N/A	Microcontroller used to control All smart devices and sensors.
	Flutter Framework	3.13	Enables cross-platform mobile app development with responsive UI.
	Breadboard	N/A	Used for prototyping and testing circuit connections.
	Firebase	Latest	Provides authentication and real-time communication between app and ESP32.
	Sensors	N/A	DHT11, LDR, PIR, Water Level Sensor

1.11. Work Division

To ensure smooth development and clear responsibility distribution, the project tasks have been divided among team members based on their skills and areas of expertise. The work division is summarized in **Table 9**.

Table 9: Work Division

Sr. No	Roll Number	Name	Role Assignment & Work Division
1.	BCS22006	Muhammad Mohsin Saleem	Mobile App Development: Flutter UI/UX, Firebase integration, Wi-Fi device control screens
2.	BCS22016	Abdul Rehman Tahir	Project Lead / ESP32 development: Sensor integration, relay control, Automatic Mode logic
3.	BCS22021	Rehan Ali	Backend & DevOps: Firebase configuration, database structure, GitHub management
4.	BCS22022	Muhammad Owais	ESP32 Hardware: Circuit Design, sensor wiring, relay modules, ESP32 configuration
5.	BCS22024	Wasif Ali	Mobile app development: Flutter application design, device-control modules, real-time updates, app testing

1.12. Conclusion

This chapter presented the foundation of the Smart Home Automation System by outlining the overall project proposal, including its motivation, goals, scope, and expected outcomes. The problem statement, objectives, and success criteria established a clear direction for the system, ensuring that the solution remains aligned with user needs and practical constraints. Key assumptions and potential risks were also identified to provide a realistic understanding of the project environment.

Overall, Chapter 01 defined *what* the system aims to achieve and *why* it is being developed. This groundwork serves as the basis for the detailed requirements and design decisions discussed in the subsequent chapters, guiding the development of a reliable, scalable, and user-focused smart home automation solution.

Chapter No 2 Literature Review

2.1 Literature Survey

Existing research in smart home automation highlights the growing use of IoT technologies to enable remote device control, environmental monitoring, and automated decision-making. Prior systems commonly utilize microcontrollers such as Arduino, ESP8266, and ESP32, paired with sensors like DHT11/DHT22, LDRs, and PIR modules to collect real-time environmental data. Studies also show that cloud platforms, including Firebase, MQTT brokers, and REST APIs—are widely adopted for enabling remote access and synchronization across devices.

Many implementations provide basic automation, such as temperature-based fan control or light-dependent switching. However, several limitations are frequently observed: lack of cross-platform mobile interfaces, limited scalability, dependence on local networks, and minimal integration of real-time cloud services. Recent work emphasizes the importance of user-friendly mobile applications, with frameworks like Flutter gaining popularity for their cross-platform support and responsive UI.

The reviewed literature indicates that while numerous smart home solutions exist, there remains a need for systems that combine **real-time cloud connectivity, modular automation logic, and a unified mobile interface**. This project builds on these findings by integrating Flutter, Firebase Realtime Database, and ESP32 to deliver a scalable and responsive smart home automation prototype.

2.2 Related Work

Research highlights the importance of IoT-based automation for energy efficiency and convenience. Research highlights that microcontrollers like the ESP32 can reliably control appliances and communicate with cloud platforms through Wi-Fi (Ahmad, 2023). Sensor-based automation using temperature, motion, and light sensors is widely used to reduce manual effort and support smart decision-making (Nguyen, Patel, & Kumar, 2025).

Commercial systems such as Google Nest and Amazon Alexa offer advanced features but are expensive and often require proprietary hardware, making them less suitable for low-cost implementations (Kaur, 2020).

2.3 Gap Analysis

Existing solutions still have limitations:

- Many low-cost systems lack multi-sensor automation
- Limited support for timer-based control and peak-time alerts
- Commercial systems are costly and not easily customizable
- Few systems offer subscription-based analytics

This project fills these gaps by providing a **low-cost, ESP32-based smart home system** with sensor automation, timer control, peak-time alerts, and a subscription-enabled analytics model.

2.4 Summary

The literature reviewed in this chapter shows that IoT-based smart home automation has been widely explored using microcontrollers, sensors, and cloud platforms to enable remote control and automated decision-making. Prior research demonstrates the effectiveness of devices like the ESP32 and sensors such as DHT, LDR, and PIR for building low-cost automation systems, while highlighting the growing role of cloud services and cross-platform mobile applications.

Existing academic and commercial solutions offer useful features but also present limitations. Research prototypes often lack scalability, real-time cloud integration, and unified mobile interfaces, whereas commercial systems like Google Nest and Amazon Alexa are costly, rely on proprietary hardware, and offer limited customization.

The gap analysis indicates a clear need for a system that is affordable, customizable, and capable of multi-sensor automation with real-time connectivity. The proposed project addresses these gaps by integrating ESP32 hardware, Firebase Realtime Database, and a Flutter-based mobile application to deliver a flexible and cost-effective smart home automation solution.

Chapter No 3

Software Requirements Specification

3.1 Requirements Analysis

This section summarizes the work carried out during the Software Requirements Specification (SRS) phase. The SRS defined the functional and non-functional requirements of the smart home automation system, identified all system actors, and documented the expected behaviors through detailed use cases. It established a clear understanding of what the system must achieve, the constraints under which it operates, and the interactions between the user, mobile application, cloud database, and ESP32 hardware. This analysis forms the basis for the system's design and implementation.

3.2 User classes and characteristics

This section identifies the primary user groups who will interact with the system. Understanding these users helps shape the interface design, automation features, and overall usability. Since the system relies on mobile control and cloud-based communication, it supports a wide range of users with varying technical backgrounds.

Table 10: User Class and Characteristics

User Class	User Characteristics
Homeowner	Sets up devices, manages automation modes, and monitors home appliances through the mobile app.
Family Member	Daily user who controls appliances through the mobile app; no technical setup required.
Renter	Uses plug-and-play Wi-Fi devices and prefers simple, portable automation without complex installation.
Tech Enthusiast	Interested in experimenting with automation features, sensor-based behaviour, and custom ESP32 configurations.
Small Business Owner	Uses the system for basic lighting, fan, or pump control with optional automation for energy savings.

3.3 Requirement Identifying Technique

The requirements for the system are gathered using structured techniques to ensure clarity, completeness, and alignment with real-world usage. Since the system supports mobile control and sensor-based automation, a systematic approach is essential.

3.3.1 Primary Technique: Use Cases

Use cases describe how users interact with the system to achieve specific goals, such as controlling a device, enabling Automatic Mode, or viewing real-time status. They provide a clear, scenario-based understanding of system behaviour and are widely used for communicating requirements among stakeholders. Use

cases remain a widely adopted approach for requirement structuring due to clarity in stakeholder communication (Sommerville, 2016).

3.3.2 Supporting Technique: User Stories

User stories capture requirements from the user’s perspective in simple, natural language (e.g., “*As a user, I want to automatically turn on the fan when the room gets hot.*”). These stories help ensure the system remains practical, intuitive, and aligned with everyday usage patterns. They also support interface design and automation logic development. User-story method encourages requirement realism focused on everyday usage (Sommerville, 2016).

3.4 Functional Requirements

This section defines the functional behaviour of the IoT-Based Smart Home Automation System. After registering and logging in through Firebase Authentication, users can control all Wi-Fi-enabled appliances connected to the ESP32, monitor real-time device states, and enable Automatic Mode for sensor-based automation. The system follows modern cloud-connected smart-home models, enabling remote control, real-time synchronization, and intelligent automation. Features align with cloud-connected smart-home models enabling remote appliance control and energy automation (Ahmad, 2023; Sharma, 2022; Nguyen, Patel, & Kumar, 2025).

3.4.1 User Registration (Functional Requirement: FR-1)

This feature allows a new user to create an account using Firebase Authentication. As shown in **Table 11**, the system verifies the entered information and creates a unique user profile if all conditions are met.

Table 11 Functional Requirement – User Registration

Identifier	FR-1
Title	User Registration
Requirement	When a new user provides a valid email and password, the system shall validate the information and create a unique user account using Firebase Authentication.
Source	Project Objective: Provide secure user registration and login through Firebase Authentication.
Inputs	Email, Password
Destination	User profile stored in Firebase Authentication; confirmation displayed in the app
Outputs	Successful registration message; error message if validation fails

Rationale	Ensures that only authenticated users can access and control smart devices.
Business Rule (if required)	Password must be at least 8 characters long and contain at least one letter and one number.
Dependencies	--
Priority	High

3.4.2 User Login (Functional Requirement: FR-2)

This feature enables registered users to access their account using correct login credentials. As shown in *Table 12*, the system verifies details before granting access.

Table 12: Functional Requirement - User login

Identifier	FR-2
Title	User Login
Requirement	The system shall allow a registered user to log in by validating their email and password through Firebase Authentication.
Source	Derived from FR-1
Inputs	Email, Password
Destination	Firebase Authentication; user dashboard
Outputs	Login success message; error message for invalid credentials
Rationale	Ensures secure access to device control and automation features.
Business Rule (if required)	After 3 failed login attempts, the account shall be locked for 5 minutes.
Dependencies	FR-1
Priority	High

3.4.3 Device Onboarding (Functional Requirement: FR-3)

This requirement enables users to add ESP32-controlled appliances to their account. As shown in *Table 13*, devices must be connected to the same Wi-Fi network during onboarding.

Table 13: Functional Requirement - Device Onboarding

Identifier	FR-3
Title	Device Onboarding (Wi-Fi Devices)
Requirement	The system shall allow a logged-in user to add ESP32-controlled appliances through a guided setup process.
Source	Project Objective: Ensure seamless communication between the mobile app, Firebase, and ESP32 hardware.
Inputs	Device ID, Wi-Fi credentials (if required)
Destination	Device entry stored in Firebase; device added to user's device list
Outputs	Device successfully added message; error message if onboarding fails
Rationale	Devices must be registered before they can be controlled or automated.
Business Rule (if required)	Devices must be connected to the same Wi-Fi network during onboarding.
Dependencies	FR-2
Priority	High

3.4.4 Remote Appliance ON/OFF Control (Functional Requirement: FR-4)

This requirement allows users to manually control appliances through the mobile app. Commands are sent to the ESP32 via Firebase. As shown in **Table 14**, the command is sent.

Table 14: Functional Requirement - Device On/Off

Identifier	FR-4
Title	Remote Appliance Control (ON/OFF)
Requirement	The system shall allow the user to turn any connected ESP32-controlled appliance ON or OFF using the mobile application.
Source	Project Objective: Implement remote ON/OFF control for all connected devices.
Inputs	Device selection, ON/OFF command
Destination	Firebase Realtime Database → ESP32
Outputs	Updated device state displayed in the app
Rationale	Core functionality for smart home management.
Business Rule (if required)	If a device is offline, the app shall display an appropriate status message.
Dependencies	FR-3
Priority	High

3.4.5 Automatic Mode Activation (Functional Requirement: FR-5)

This requirement enables users to activate Automatic Mode, allowing the ESP32 to make decisions based on sensor inputs as shown in **Table 15**.

Table 15: Functional Requirement - Automatic Mode

Identifier	FR-5
-------------------	------

Title	Automatic Mode Activation
Requirement	The system shall allow the user to enable or disable Automatic Mode from the mobile application.
Source	Project Objective: Introduce Automatic Mode for environment-based decision making.
Inputs	Automatic Mode toggle
Destination	Firebase automation flag; ESP32 automation logic
Outputs	Automatic Mode status displayed in the app
Rationale	Allows the ESP32 to automatically control appliances using sensor inputs.
Business Rule (if required)	Automatic Mode overrides manual ON/OFF commands for supported appliances.
Dependencies	FR-3
Priority	High

3.4.6 Temperature-Based Fan Control (Functional Requirement: FR-6)

This requirement defines how the system uses temperature readings to automate fan control when Automatic Mode is active as shown in *Table 16*.

Table 16: Functional Requirement - Temperature-Based Fan Control

Identifier	FR-6
Title	Temperature-Based Fan Control
Requirement	The system shall automatically turn the fan ON or OFF based on temperature thresholds when Automatic Mode is active.
Source	Project Goal: Enable intelligent automation using temperature sensors.
Inputs	Temperature sensor readings; threshold values

Destination	Firestore → ESP32 fan control
Outputs	Automated fan ON/OFF action
Rationale	Improves comfort and energy efficiency.
Business Rule (if required)	Threshold values may be predefined or user-adjustable.
Dependencies	FR-5
Priority	High

3.4.7 Motion-Based Light Automation (Functional Requirement: FR-7)

This requirement describes how the system uses motion detection to automate lighting as shown in **Table 17**.

Table 17: Functional Requirement - Motion Based Light Automation

Identifier	FR-7
Title	Motion-Based Light Automation
Requirement	The system shall automatically turn lights ON or OFF based on motion detection when Automatic Mode is active.
Source	Project Goal: Enable automation using PIR motion sensors.
Inputs	PIR sensor readings
Destination	Firestore → ESP32 light control
Outputs	Automated light ON/OFF action
Rationale	Enhances convenience and reduces unnecessary energy usage.
Business Rule (if required)	Uses PIR sensor for occupancy detection.

Dependencies	FR-5
Priority	Medium

3.4.8 Light-Level Curtain Automation (Functional Requirement: FR-8)

As shown in **Table 18**, this requirement explains how the system uses ambient light levels to automate curtain movement.

Table 18: Functional Requirement - Light level Curtain Automation

Identifier	FR-8
Title	Light-Level Curtain Automation
Requirement	The system shall automatically open or close curtains based on ambient light levels when Automatic Mode is active.
Source	Project Goal: Enable automation using LDR sensors.
Inputs	LDR sensor readings
Destination	Firebase → ESP32 curtain motor control
Outputs	Curtain open/close action
Rationale	Improves comfort and natural lighting efficiency.
Business Rule (if required)	Uses LDR sensor to detect day/night conditions.
Dependencies	FR-5
Priority	Medium

3.4.9 Water-Level Pump Automation (Functional Requirement: FR-9)

This requirement defines how the system manages water pump operation based on tank water levels as shown in **Table 19**.

Table 19: Functional Requirement - Water Pump Automation

Identifier	FR-9
Title	Water-Level Pump Automation
Requirement	The system shall automatically turn the water pump ON or OFF based on tank water level readings when Automatic Mode is active.
Source	Project Goal: Enable automation using water level sensors.
Inputs	Water level sensor readings
Destination	Firebase → ESP32 pump control
Outputs	Pump ON/OFF action
Rationale	Prevents tank overflow and ensures efficient water usage.
Business Rule (if required)	Uses water level sensor to detect minimum and maximum thresholds.
Dependencies	FR-5
Priority	High

3.4.10 Peak-Time Buzzer Alert (Functional Requirement: FR-10)

This requirement specifies how the system alerts users during predefined peak-time intervals as shown in **Table 20**.

Table 20: Functional Requirement- Peak-Time Buzzer Alert

Identifier	FR-10
Title	Peak Time Buzzer Alert

Requirement	The system shall activate a buzzer during predefined peak time intervals.
Source	Project Goal: Provide user alerts based on time-based events.
Inputs	Time schedule; peak time settings
Destination	Firebase → ESP32 buzzer control
Outputs	Buzzer alert
Rationale	Helps users manage energy usage during peak hours.
Business Rule (if required)	Peak time ranges may be predefined or user-adjustable.
Dependencies	FR-3
Priority	Low

3.4.11 Communication Handling (Functional Requirement: FR-11)

This requirement defines how the system manages communication between the mobile app, Firebase, and ESP32 as shown in *Table 21*.

Table 21: Functional Requirement - Communication Handling

Identifier	FR-11
Title	Communication Handling
Requirement	The system shall use Firebase Authentication for login and shall communicate with ESP32 devices using Wi-Fi and Firebase Realtime Database.
Source	Project Objective: Ensure seamless communication between the mobile app, Firebase, and ESP32 hardware.
Inputs	User commands; sensor data
Destination	Firebase Authentication; Firebase Realtime Database; ESP32

Outputs	Synchronized device states and sensor updates
Rationale	Ensures secure login and reliable device communication.
Business Rule (if required)	Device control shall occur through Firebase-synchronized commands.
Dependencies	FR-1, FR-2
Priority	High

3.4.12 Subscription Management (Functional Requirement: FR-12)

This requirement defines how the system handles subscription-based access. Users may have access to certain features only if they are subscribed. The system must allow users to view their subscription status and upgrade when needed as shown in *Table 22*.

Table 22: Functional Requirement - Subscription Management

Identifier	FR-12
Title	Subscription Management
Requirement	The system shall allow users to view their current subscription status and upgrade to a higher plan if required.
Source	Project Objective: Provide controlled access to selected features based on subscription level.
Inputs	User ID; subscription selection
Destination	Firebase subscription node
Outputs	Subscription status display; upgrade confirmation
Rationale	Ensures that premium features are accessible only to subscribed users.
Business Rule (if required)	Subscription status must be verified through Firebase before granting access to restricted features.

Dependencies	FR-1, FR-2
Priority	Medium

3.5 Non-Functional Requirements

This section defines the non-functional requirements that support the functional behavior of the smart home automation system. These requirements describe system qualities such as reliability, usability, performance, and security.

3.5.1 Reliability

- The system must operate consistently with minimal failures to ensure dependable smart home operation.
- The system shall maintain a **Mean Time Between Failures (MTBF)** of at least **30 days** under normal usage.
- The system shall recover from temporary communication failures (Wi-Fi or Firebase) within **5 seconds** once connectivity is restored.
- Sensor readings and device states shall remain consistent, ensuring **at least 95% data integrity** during transmission.
- In case of failure, the system shall log the error and retry the operation automatically.
- Critical operations (e.g., automation triggers) shall include fallback checks to prevent unsafe device behaviour.

3.5.2 Usability

- The system must be easy to learn, intuitive to use, and accessible for users with varying technical backgrounds.
- A new user shall be able to complete registration and login within **3 steps**.
- The mobile application shall allow users to control any device with **no more than 2 taps** from the dashboard.
- All icons, labels, and controls shall follow a consistent design pattern to reduce user confusion.
- Error messages shall be clear, descriptive, and guide the user toward corrective action.
- The interface shall remain usable on screens ranging from **5-inch to 7-inch** mobile devices.

3.5.3 Performance

- The system must respond quickly and efficiently to user actions and sensor events.
- Device ON/OFF commands shall be reflected in the app within 2 seconds.
- Sensor data updates shall appear in the dashboard within 3 seconds of being generated by the ESP32.
- The mobile application shall load the dashboard within **5 seconds** after login.

- Automation rules (temperature, motion, light, water level) shall execute within **1 second** of receiving sensor input.
- The system shall support at **least 10 connected devices** without performance degradation.

3.5.4 Security

- The system must protect user data, device states, and communication channels from unauthorized access.
- Only authenticated users shall be able to access device controls and automation features.
- All communication between the mobile app, Firebase, and ESP32 shall be protected against unauthorized modification.
- User data shall be stored in a secure environment with restricted access.
- Subscription-based features shall only be accessible after verifying the user's subscription status.
- The system shall prevent unauthorized device onboarding by validating device identity before registration.

3.6 External Interface Requirements

This section describes how the smart home automation system interacts with users, external hardware, and software components. These interfaces ensure proper communication between the mobile application, Firebase services, and ESP32-based IoT devices.

3.6.1 User Interfaces Requirements

The system provides a mobile application that serves as the primary interface between the user and the smart home environment. The UI must be simple, intuitive, and consistent with modern Android design guidelines to ensure accessibility and ease of use. References to GUI standards or product family style guides that are to be followed.

- **Look and Feel:** The application shall follow standard Android UI guidelines, using a clean colour palette, readable typography, and consistent spacing.
- **Layout:** The main dashboard shall display all ESP32-connected devices. Selecting a device opens its control screen, where users can toggle ON/OFF, view status, or configure automation settings. A menu button shall provide access to settings and logout.
- **Standard Items:** Each screen shall include a back button and clearly visible controls such as ON/OFF toggles, Automatic Mode switches, and threshold input fields.
- **Notifications:** System alerts (e.g., Automatic Mode activated, low water level) shall be delivered using standard Android push notifications.
- **Localization and Clarity:** The UI shall be designed to support easy translation into other languages in future versions.

3.6.2 Software Interfaces

The smart home automation system interacts with several external software components to support authentication, data storage, device communication, and notification delivery. The system uses **Firestore Authentication** to manage user registration, login, and identity verification. All user credentials are processed through Firestore's secure API endpoints to ensure controlled access to device features.

The system relies on **Firestore Realtime Database or Firestore** to store device metadata, automation settings, user profiles, and real-time device states. These databases act as the communication bridge between the mobile application and ESP32 devices, using structured JSON data for consistency and interoperability.

The mobile application operates on the **Android operating system** (version 8.0 or above) and uses standard Android libraries for UI rendering, network communication, and background services. The application is developed using the Flutter framework, which provides cross-platform UI components and ensures consistent behaviour across supported devices.

Communication between the mobile application and ESP32 hardware occurs indirectly through Firestore. The ESP32 connects to Wi-Fi and synchronizes device states, sensor readings, and automation commands using **HTTP** or **MQTT** protocols, depending on the firmware implementation. All exchanged data follows JSON formatting to maintain compatibility across components.

The system also uses **ESP32** firmware libraries for Wi-Fi connectivity, GPIO control, sensor reading, and Firestore communication, enabling the microcontroller to process automation logic and respond to commands issued from the mobile application.

3.6.3 Hardware Interfaces

The system interacts with physical devices through the ESP32 microcontroller, which serves as the central hardware interface. The ESP32 receives sensor inputs and sends control signals to connected appliances using its GPIO and ADC pins. Supported hardware includes relay modules for switching appliances, sensors such as DHT11, LDR, PIR, and water-level sensors for environmental monitoring, and a buzzer for alert notifications. All device commands and sensor updates are processed by the ESP32 and synchronized with the software system through Wi-Fi communication.

3.6.4 Communications Interfaces

The smart home automation system relies on cloud-based and local network communication to exchange data between the mobile application, Firestore services, and ESP32 hardware. All communication between the mobile app and Firestore uses HTTPS to ensure secure data transfer. The ESP32 connects to the local Wi-Fi network and synchronizes device states, sensor readings, and automation commands using either HTTP or MQTT, depending on the firmware implementation. Data exchanged across all components follows a JSON format to maintain consistency and interoperability.

The system requires an active internet connection for authentication, database synchronization, and notification delivery. During initial device onboarding, the ESP32 must be connected to the same Wi-Fi network as the mobile device. After onboarding, communication occurs through the cloud, allowing the user and ESP32 to operate on different networks without affecting functionality.

3.7 Use case Analysis

This section breaks down the main ways users will interact with the system. Each "use case" is a specific goal a user wants to achieve.

3.7.1 Use Case #1: User Registration (UC-1)

This use case allows a new user to create an account by providing their basic credentials so they can securely access the system as shown in **Figure 3 & Table 23**.

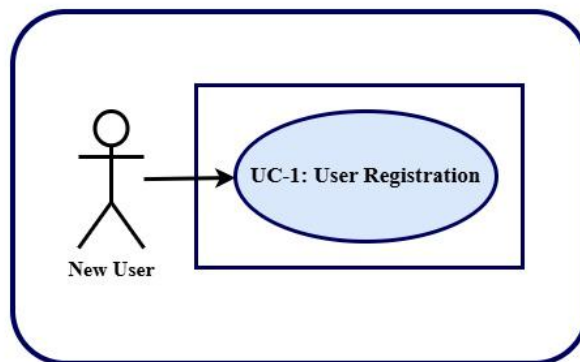


Figure 3: User Registration

Following is the table for UC-1:

Table 23: Use Case Description for "User Registration"

UC Identifier	UC-1
Requirements Traceability	FR-1
Purpose	Allow a new user to create an account securely.
Priority	High
Preconditions	App installed; user not logged in.
Post conditions	Account created in Firebase.
Actors	New User
Extends	--
Main Success Scenario	• User opens app → selects "Sign Up". • Enters email + password. • System validates credentials. • Account created. • User redirected to Login screen.

Alternate Flows	A1: User enters weak password - app suggests stronger password.
Exceptions	E1: Email already registered - shows error.
Includes	--

3.7.2 Use Case #2: Login to the System (UC-2)

This use case allows a registered user to sign into the system by entering valid login credentials to access their connected devices and app features as shown in **Figure 4 & Table 24**.

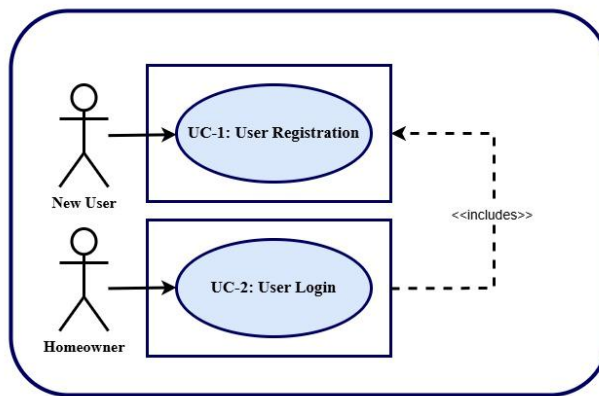


Figure 4: User Login

Following is the table for UC-2:

Table 24: Use Case Description for "User Login"

UC Identifier	UC-1
Requirements Traceability	FR-1
Purpose	Allow a new user to create an account securely.
Priority	High
Preconditions	App installed; user not logged in.
Post conditions	Account created in Firebase.
Actors	New User
Extends	--
Main Success Scenario	• User opens app → selects “Sign Up”. • Enters email + password. • System validates credentials. • Account created. • User redirected to Login screen.
Alternate Flows	A1: User enters weak password - app suggests stronger password.
Exceptions	E1: Email already registered - shows error.
Includes	--

3.7.3 Use Case #3: Add/Pair a New Device (ESP32) (UC-3)

This use case enables the user to connect a new smart appliance (ESP32) to their account through a guided setup process in the mobile app as shown in **Figure 5** & **Table 25**.

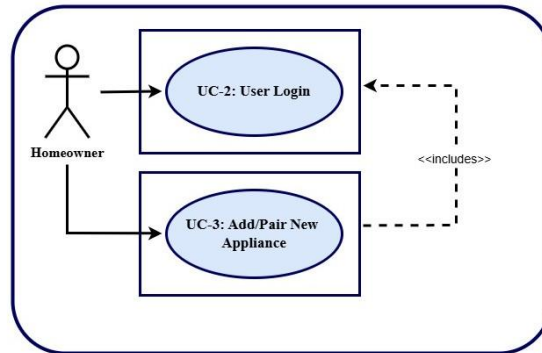


Figure 5: Add/Pair a New Device

Following is the table for UC-3:

Table 25: Use Case Description for "Adding/Pairing a new appliance"

UC Identifier	UC-3
Requirements Traceability	FR-3
Purpose	Allow user to add ESP32-controlled devices.
Priority	High
Preconditions	User logged in; ESP32 powered and connected to Wi-Fi.
Post conditions	Device appears in dashboard.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User selects Add Device. • System retrieves available devices from Firebase. • User selects device. • Device linked to account. • Device added to dashboard.
Alternate Flows	A1: User enters Device ID manually.
Exceptions	E1: Device not found - prompts user to retry.
Includes	UC-2

3.7.4 Use Case #4: Control Device ON/OFF (UC-4)

This use case allows the user to switch a connected Wi-Fi appliance ON or OFF using the mobile application as shown in **Figure 6 & Table 26**.

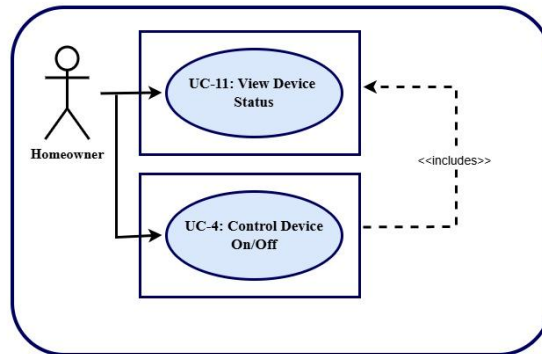


Figure 6: Control Device ON/OFF

Following is the table for UC-4:

Table 26: Use Case Description for " Controlling Device "

UC Identifier	UC-4
Requirements Traceability	FR-4
Purpose	Allow user to toggle appliances ON/OFF.
Priority	High
Preconditions	Device paired and reachable.
Post conditions	Device state updated.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	▪ User taps toggle button. ▪ Command sent to Firebase. ▪ ESP32 receives update. ▪ Device switches state. ▪ App updates status.
Alternate Flows	--
Exceptions	E1: Device unreachable
Includes	UC-11 (viewing device status)

3.7.5 Use Case #5: Activate Automatic Mode (UC-5)

This use case allows the user to enable or disable Automatic Mode as shown in **Figure 7** and **Table 27**.

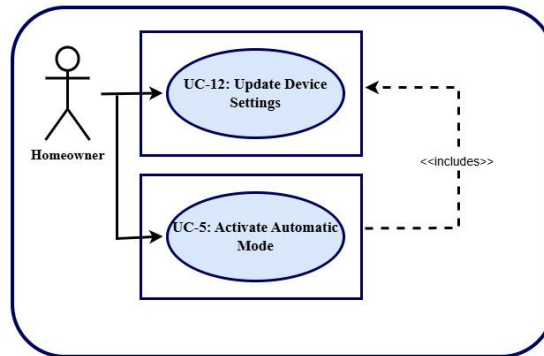


Figure 7: Automatic Mode Activation

Following is the table for UC-5:

Table 27: Use Case Description for " Automatic Mode Activation "

UC Identifier	UC-5
Requirements Traceability	FR-5
Purpose	Allow user to enable/disable Automatic Mode.
Priority	High
Preconditions	Device paired; sensors connected.
Post conditions	Automatic Mode activated.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Toggles “Automatic Mode” • Firebase updates mode. • ESP32 begins automation.
Alternate Flows	--
Exceptions	E1: ESP32 not responding.
Includes	UC-12 (when adjusting thresholds)

3.7.6 Use Case #6: Temperature-Based Fan Automation (UC-6)

This use case describes how the ESP32 automatically controls the fan based on temperature.as shown in **Figure 8** and **Table 28**.

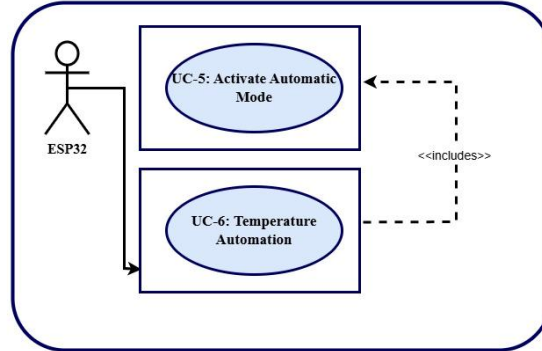


Figure 8: Temperature-Based Fan Automation

Following is the table for UC-6:

Table 28: Use Case Description for " Temperature-Based Fan Automation "

UC Identifier	UC-6
Requirements Traceability	FR-6
Purpose	Automatically control fan based on temperature.
Priority	High
Preconditions	Automatic Mode enabled; DHT11 connected.
Post conditions	Fan ON/OFF based on threshold.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads temperature. • If temperature > threshold → Fan ON. • If temperature < threshold → Fan OFF.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

3.7.7 Use Case #7: Motion-Based Light Automation (UC-7)

This use case describes how the ESP32 controls lights based on motion detection as shown in **Figure 9** & **Table 29**.

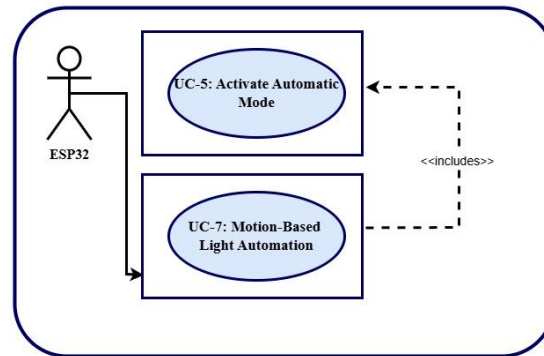


Figure 9: Motion-Based Light Automation

Following is the table for UC-7:

Table 29: Use Case Description for " Motion-Based Light Automation "

UC Identifier	UC-7
Requirements Traceability	FR-7
Purpose	Automatically control lights based on motion.
Priority	Medium
Preconditions	Automatic Mode enabled; PIR sensor connected.
Post conditions	Lights ON/OFF automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 detects motion. • Light turns ON. • No motion for set duration → Light OFF.
Alternate Flows	--
Exceptions	E1: Sensor not responding.
Includes	UC-5

3.7.8 Use Case #8: Light-Level Curtain Automation (UC-8)

This use case describes how the ESP32 automates curtains based on ambient light.as shown in **Figure 10 & Table 30**.

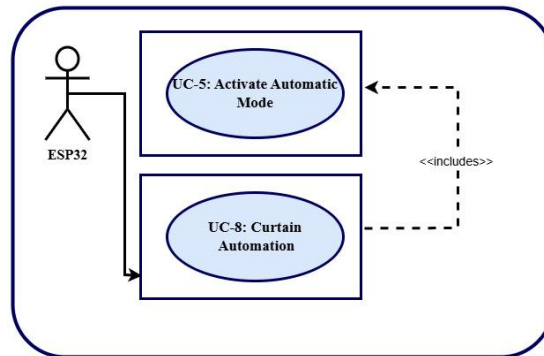


Figure 10: Light-Level Curtain Automation

Following is the table for UC-8:

Table 30: Use Case Description for " Light-Level Curtain Automation "

UC Identifier	UC-8
Requirements Traceability	FR-8
Purpose	Automatically open/close curtains based on light level.
Priority	Medium
Preconditions	Automatic Mode enabled; LDR connected.
Post conditions	Curtains open/close automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads light level. • Low light → Curtains open. • High light → Curtains close.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

3.7.9 Use Case #9: Water-Level Pump Automation (UC-9)

This use case describes how the ESP32 controls the water pump based on tank level.as shown in **Figure 11** and **Table 31**.

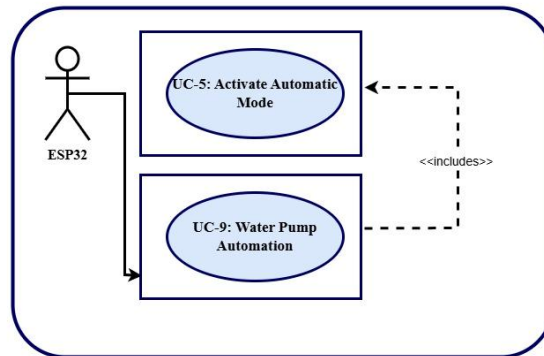


Figure 11: Water-Level Pump Automation

Following is the table for UC-9:

Table 31: Use Case Description for " Water-Level Pump Automation "

UC Identifier	UC-9
Requirements Traceability	FR-9
Purpose	Automatically control pump based on water level.
Priority	High
Preconditions	Automatic Mode enabled; water-level sensor connected.
Post conditions	Pump ON/OFF automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads water level. • Low level → Pump ON. • High level → Pump OFF.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

3.7.10 Use Case #10: Peak-Time Buzzer Alert (UC-10)

This use case describes how the ESP32 triggers a buzzer during peak hours.as shown in **Figure 12 & Table 32**.

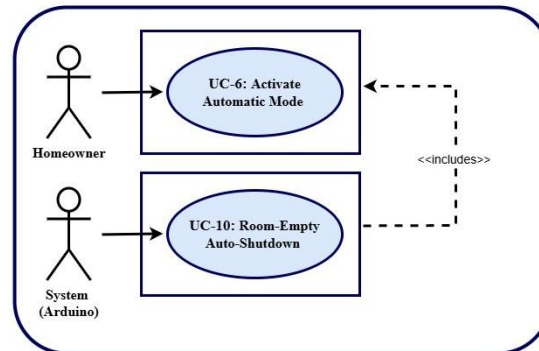


Figure 12: Peak-Time Alert

Following is the table for UC-10:

Table 32: Use Case Description for " Peak-Time Alert "

UC Identifier	UC-10
Requirements Traceability	FR-10
Purpose	Notify users during peak-time intervals.
Priority	Low
Preconditions	ESP32 connected; peak-time configured.
Post conditions	Buzzer activated.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• System clock reaches peak time. • ESP32 activates buzzer. • App displays notification.
Alternate Flows	--
Exceptions	E1: Buzzer not responding.
Includes	-

3.7.11 Use Case #11: View Device Status (UC-11)

This use case allows user to view real-time status of all devices as shown in **Figure 13 & Table 33**.

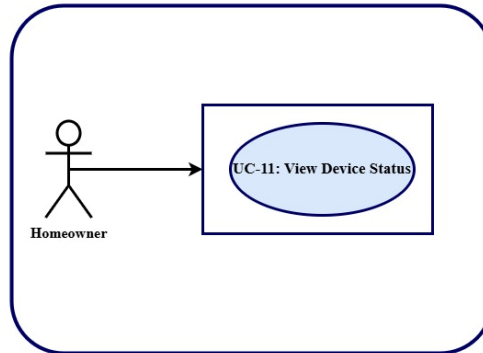


Figure 13: View Device Status

Following is the table for UC-11:

Table 33: Use Case Description for " View Device Status"

UC Identifier	UC-11
Requirements Traceability	FR-11
Purpose	Display real-time device states.
Priority	Medium
Preconditions	Devices paired.
Post conditions	Status displayed.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens dashboard. • App fetches device states from Firebase. • States displayed.
Alternate Flows	--
Exceptions	E1: Device unreachable.
Includes	--

3.7.12 Use Case #12: Update Device Settings (UC-12)

This use case allows the user to update automation thresholds and settings.as shown in **Figure 14 & Table 34**.

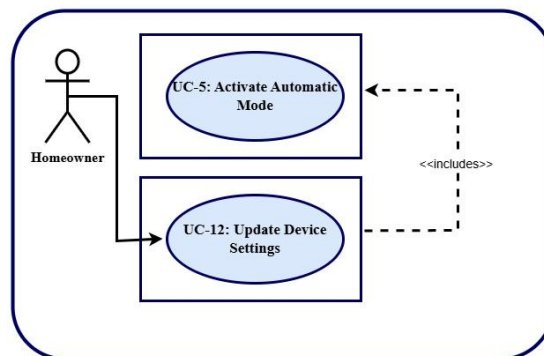


Figure 14: Update Device Settings

Following is the table for UC-12:

Table 34: Use Case Description for "Update Device Settings"

UC Identifier	UC-12
Requirements Traceability	FR-5, FR-6, FR-7, FR-8, FR-9
Purpose	Allow user to update thresholds and automation settings.
Priority	Medium
Preconditions	Device paired; user logged in.
Post conditions	New settings saved and applied.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Adjusts thresholds or modes. • App sends updated settings to Firebase. • ESP32 applies new configuration.
Alternate Flows	--
Exceptions	E1: Invalid input values.
Includes	UC-5

3.7.13 Use Case #13: Manage Subscription (UC-13)

This use case allows the user to view their subscription status and upgrade to a higher plan if required. It does not introduce new system features; it simply manages access to existing ones based on subscription level as shown in **Figure 15** & **Table 35**.

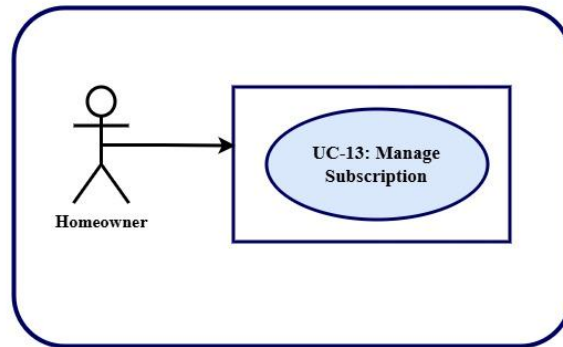


Figure 15: Manage Subscription

Following is the table for UC-13:

Table 35: Use Case Description for " Manage Subscription "

UC Identifier	UC-12
Requirements Traceability	FR-5, FR-6, FR-7, FR-8, FR-9
Purpose	Allow user to update thresholds and automation settings.
Priority	Medium
Preconditions	Device paired; user logged in.
Post conditions	New settings saved and applied.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Adjusts thresholds or modes. • App sends updated settings to Firebase. • ESP32 applies new configuration.
Alternate Flows	--
Exceptions	E1: Invalid input values.
Includes	UC-5

3.7.14 Use Case Diagram

Figure 16 describes how users interact with the IoT-based Smart Home Automation System. A new user registers an account and, once authenticated, becomes a Homeowner who can log in, add ESP32-controlled devices, and manage them through the mobile application. After onboarding, the Homeowner can toggle appliances, enable Automatic Mode, view real-time device status, and adjust automation settings. The system also performs intelligent actions such as temperature-based fan control, motion-based lighting, light-level curtain automation, water-level pump control, and peak-time buzzer alerts. These interactions collectively represent the system's functional scope.

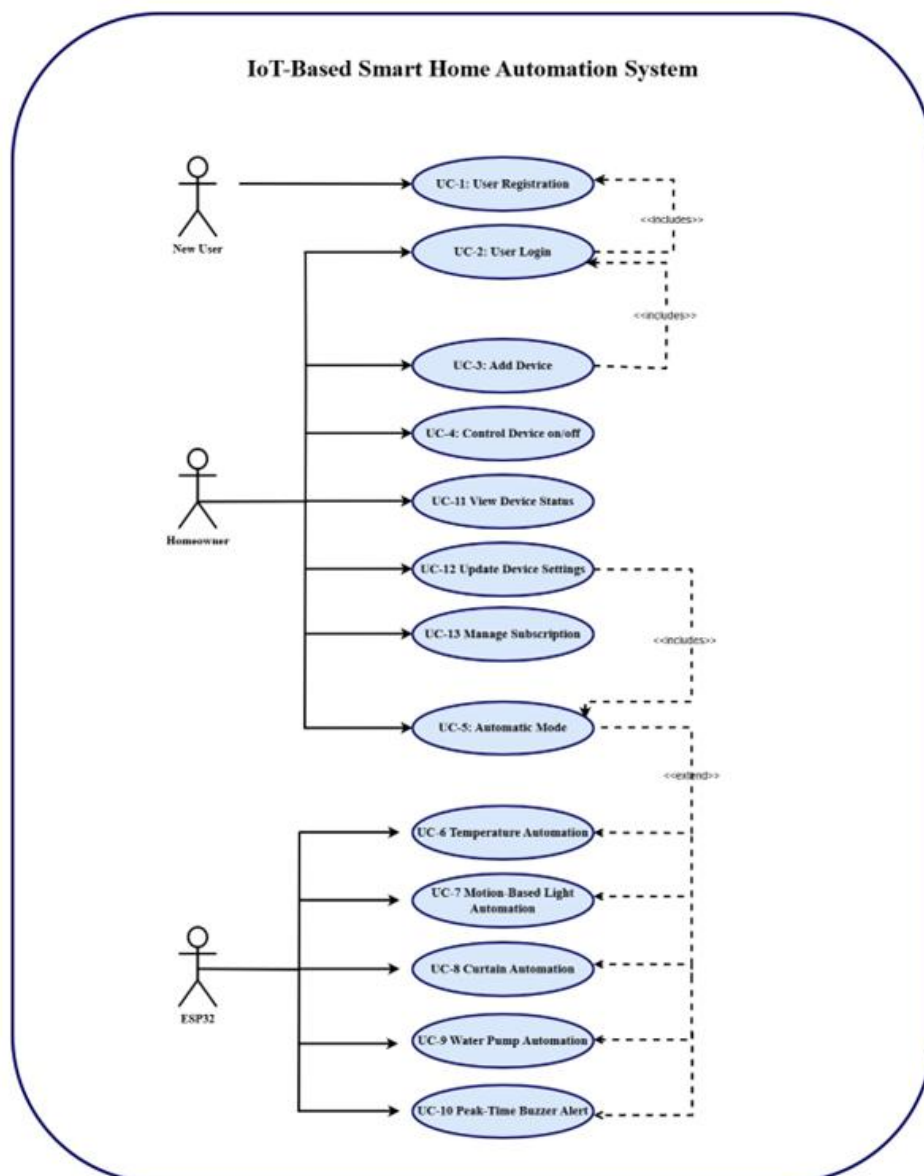


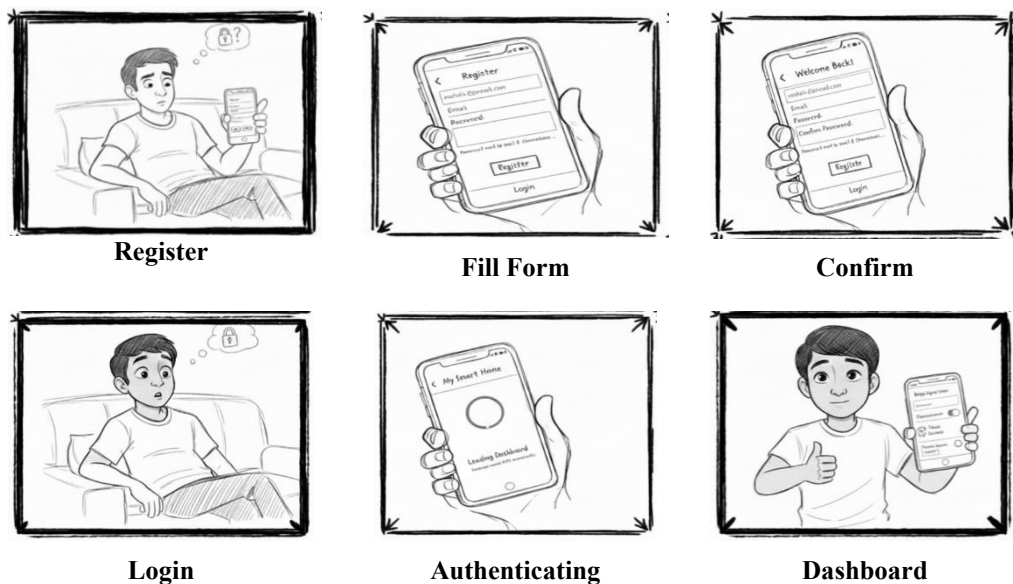
Figure 16: Use Case Diagram

3.8 Storyboards

Storyboards are sequence of drawings representing how users might use the product. They show user steps and corresponding responses by the system.

3.8.1 Storyboard 1: Registration, Login, and Dashboard Flow

The following storyboard illustrates the initial user journey within the smart home application. It focuses on the core entry-point screens, including account registration, user login, and the main dashboard interface. These screens represent the foundational steps a user completes before accessing device control, automation features, or voice commands.



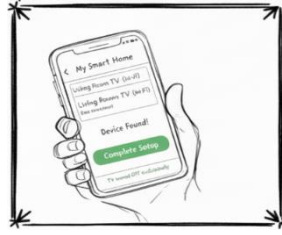
3.8.2 Storyboard 2: Device Pairing and Control

This storyboard illustrates the user journey for selecting, pairing, and controlling a Wi-Fi device (TV) through the smart home mobile application. It highlights the interaction between the mobile app and the smart device, including system feedback and user actions.





User reacts to pairing prompt



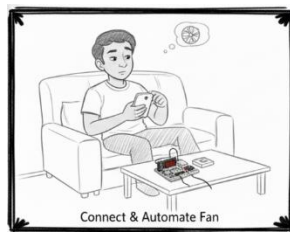
TV paired — setup complete



Control options now available

3.8.3 Storyboard 3: Fan Automation (Temperature Sensor)

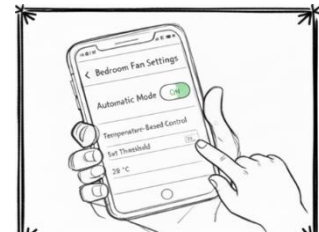
This storyboard shows how the user connects a fan through Arduino, enables Automatic Mode in the app, and lets the system adjust fan speed automatically based on room temperature.



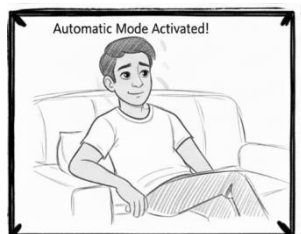
User connects fan to Arduino



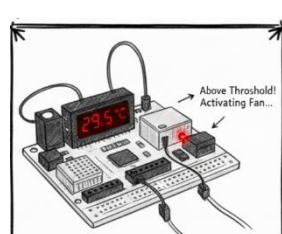
User opens dashboard



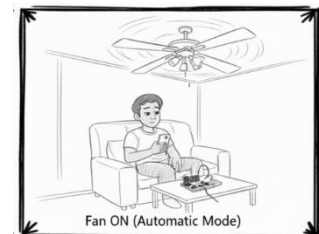
Automatic Mode enabled



Temperature exceeds threshold



Fan speed increased



Comfort maintained automatically

3.9 Summary

Requirement analysis focused on identifying the real needs of a household user and converting them into precise system specifications. Observations of daily appliance usage and informal discussions highlighted challenges such as managing multiple appliances, lack of automation, and the need for remote control. These insights shaped both the functional and non-functional requirements of the system, including Firebase-based authentication, Wi-Fi-enabled ESP32 devices, Automatic Mode, and sensor-driven automation for temperature, motion, light level, water level, and peak-time alerts.

Use case analysis structured the system's behaviour by modelling every interaction between the user and the smart home application, from registration and login to device pairing, manual control, automation configuration, and subscription management. The individual use case diagrams, scenario flows, and interaction modelling clarified how the mobile app communicates with Firebase and how ESP32 modules execute automation logic based on real-time sensor data. These structured requirements engineering approach reduced ambiguity and ensured a shared understanding among stakeholders.

The resulting specification serves as a complete blueprint for implementing a unified smart home solution. It supports remote ON/OFF control, real-time device monitoring, and fully automated behaviour through ESP32-based sensors and actuators. Through systematic requirement analysis, the project evolved from a general concept into a technically defined smart home automation architecture ready for implementation.

Chapter No 4

Software Design Specification

4.1 System Design

The system operates in a cloud-connected IoT architecture where a Flutter mobile application, Firebase services, and ESP32 microcontrollers work together to provide smart home automation. The mobile app communicates with Firebase Authentication for secure login and with Firebase Realtime Database for device commands, automation settings, and real-time status updates. ESP32 devices connect directly to the home Wi-Fi network and synchronize with Firebase to receive commands, upload sensor readings, and execute automation logic.

Each ESP32 node controls relays for appliances and reads data from sensors such as DHT11 (temperature), PIR (motion), LDR (light level), and water-level sensors. Automatic Mode is processed locally on the ESP32, ensuring fast response even when the mobile app is inactive. The design supports modular expansion, allowing additional ESP32 nodes or sensors to be added without major architectural changes.

4.2 Design Considerations

The system must maintain reliable communication between the mobile app, Firebase cloud, and ESP32 hardware. Since ESP32 devices have limited memory and processing power, the design must remain lightweight while still supporting real-time updates and automation. The architecture should be scalable, allowing multiple devices and sensors to be added easily. Security, low latency, and efficient data synchronization are key considerations.

4.2.1 Assumptions and Dependencies

- A stable internet connection is available for Firebase Authentication and Realtime Database synchronization.
- ESP32 devices can reliably connect to the home Wi-Fi network.
- Firebase services remain available for authentication and data storage.
- Users have compatible smartphones capable of running the Flutter mobile application.
- Appliances connected to ESP32 relay modules operate within safe voltage and current limits.
- Sensor modules (DHT11, PIR, LDR, water-level) function within expected accuracy ranges.

4.2.2 Limitations

- System performance depends on internet quality; slow or unstable networks may cause command delays.
- ESP32 has limited RAM and processing power, restricting extremely complex automation logic.

- Firebase free tier has usage limits that may affect large-scale deployments.
- Wi-Fi range limitations may affect ESP32 connectivity in large homes.
- Cloud latency may vary by region, affecting real-time responsiveness.

4.3 Requirements Traceability Matrix

This matrix maps each functional requirement from the SRS to the corresponding design components, modules, and hardware elements defined in the System Design Specification (SDS). It ensures that every requirement is implemented and accounted for during design. **Table 36** shows the requirement traceability matrix.

Table 36: Requirements Traceability Matrix

Requirement ID	Scope	Requirement Description	Design Specification
FR-1	User Account Creation	The system shall allow a new user to register using a valid email and password.	Authentication Module
FR-2	User Access & Authentication	The system shall allow a registered user to log in securely using Firebase Authentication.	Authentication Module
FR-3	Device Registration	The system shall allow users to add ESP32-based devices through a guided onboarding process.	Device Onboarding Module
FR-4	Manual Device Control	The system shall allow the user to remotely turn ESP32-controlled appliances ON or OFF.	Device Control Module
FR-5	Automation Configuration	The system shall allow the user to enable or disable Automatic Mode.	Automation Controller
FR-6	Intelligent Fan Automation	The system shall automatically control the fan based on temperature thresholds.	Temperature Automation Module
FR-7	Motion-Based Lighting	The system shall automatically control lights based on motion detection.	Motion Automation Module
FR-8	Light-Based Curtain Control	The system shall automatically open or close	Light Level Curtain

		curtains based on ambient light level.	Automation Module
FR-9	Water Tank Automation	The system shall automatically control the water pump based on tank water level.	Water Level Pump Automation Module
FR-10	Peak-Time Alerting	The system shall activate a buzzer during predefined peak-time intervals.	Peak Time Alert Module
FR-11	System Communication & Sync	The system shall handle communication between the mobile app, Firebase, and ESP32 devices.	Communication Handler
FR-12	Subscription & Billing	The system shall allow users to view and upgrade their subscription plan.	Subscription Management Module

4.4 Design Models

This section presents the object-oriented design models used to describe the structure and behaviour of the IoT-Based Smart Home Automation System. These models illustrate how system components interact, how data flows between them, and how the system responds to user actions and sensor events.

4.4.1 Design Class Diagram (DCD)

The class diagram represents the static structure of the smart home automation system by identifying the main classes, their attributes, methods, and relationships. It models the core entities such as **User**, **Device**, **Sensor**, **ESP32Controller**, and **AutomationManager**, along with their interactions. This diagram provides a clear blueprint for implementing the system's object-oriented architecture in both the Flutter mobile application and the ESP32 firmware.

Key classes typically include:

1. **User:** Stores authentication details and subscription status.
 - **Attributes:** userId, email, subscriptionType
 - **Functions:** register(), login(), upgradeSubscription()
2. **Device:** Represents an ESP32-controlled appliance with attributes such as deviceId, type, state, and thresholds.
 - **Attributes:** deviceId, deviceName, deviceType, state, threshold
 - **Functions:** turnOn(), turnoff(), updateStatus()

3. **Sensor:** Represents temperature, motion, light-level, or water-level sensors connected to ESP32.
 - **Attributes:** sensorId, sensorType, value\
 - **Functions:** readValue(), sendToESP32()
4. **ESP32Controller:** Handles relay control, sensor reading, and communication with Firebase.
 - **Attributes:** espId, wifiStatus
 - **Functions:** executeCommand(), readSensorData(), updateFirebase(), runAutomationLogic()
5. **AutomationManager:** Implements Automatic Mode logic based on sensor inputs and user-defined thresholds.
 - **Attributes:** autoModeEnabled
 - **Functions:** applyTemperatureLogic(), applyMotionLogic(), applyLightLevelLogic(), applyWaterLevelLogic(), applyPeakTimeLogic()
6. **FirebaseService:** Manages authentication, database reads/writes, and real-time synchronization.
 - 6.1. **Attributes:** dbReference, authReference
 - 6.2. **Functions:** authenticateUser(), writeDeviceState(), readDeviceState(), syncSensorData()
7. **SubscriptionManager:** Handles subscription validation and access control for premium features.
 - 7.1. **Attributes:** subscriptionType
 - 7.2. **Functions:** checkAccess(), upgradePlan()

Figure 17 shows the class diagram.

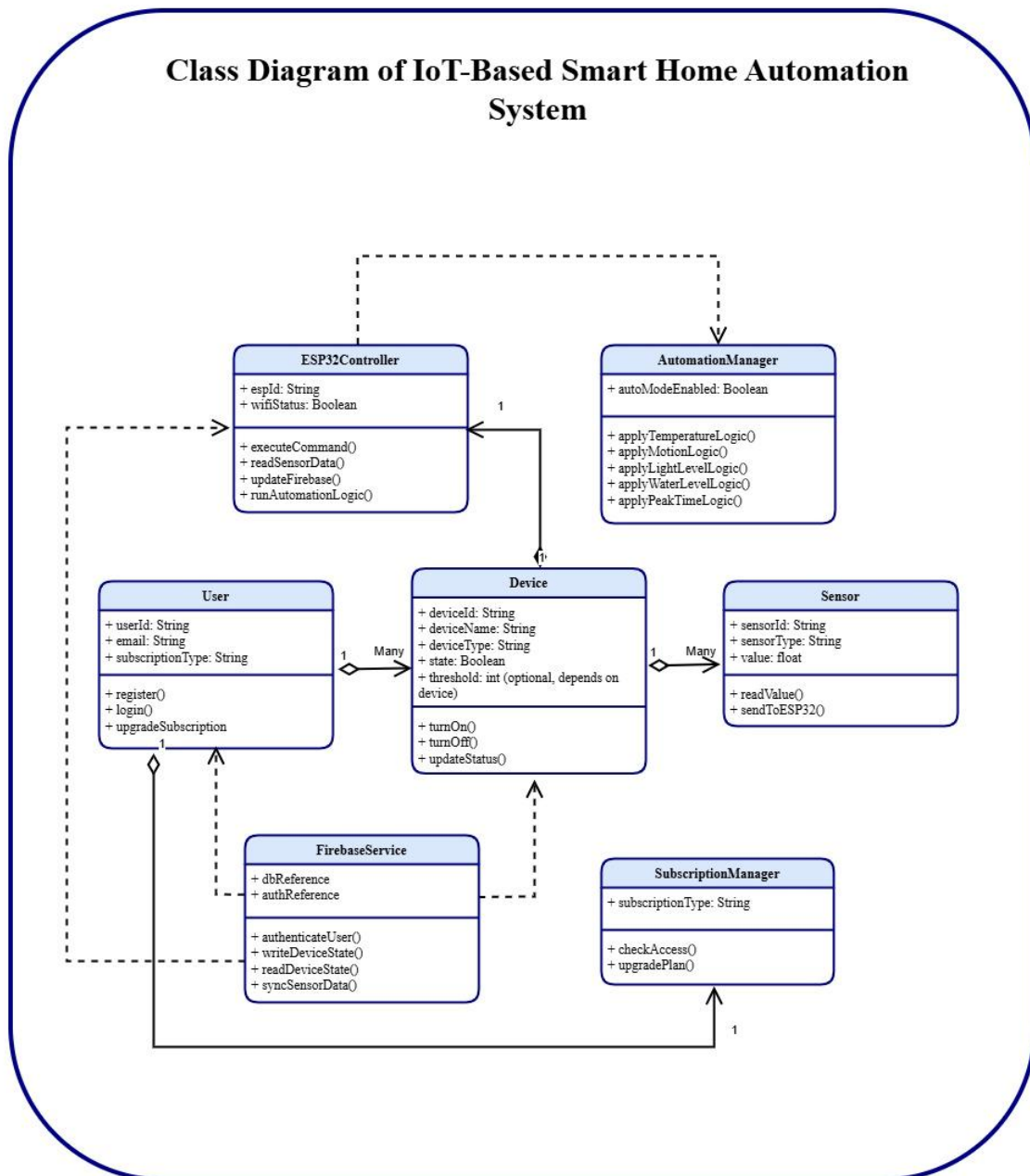


Figure 17: Class Diagram

4.4.2 Interaction Diagram (Sequence Diagram)

The sequence diagram illustrates the interaction between the User, Mobile App, Firebase Service, ESP32 Controller, and Device when the user sends a command to turn a device ON.

- **Interaction Diagram (User turns device on)**

The User initiates the action through the Mobile App, which updates the device state in Firebase. The ESP32 Controller listens for this change, activates the corresponding relay on the Device, and updates the status back to Firebase. The Mobile App then reflects the updated state to the User in real time.

Figure 18 shows the UML Interaction Diagram.

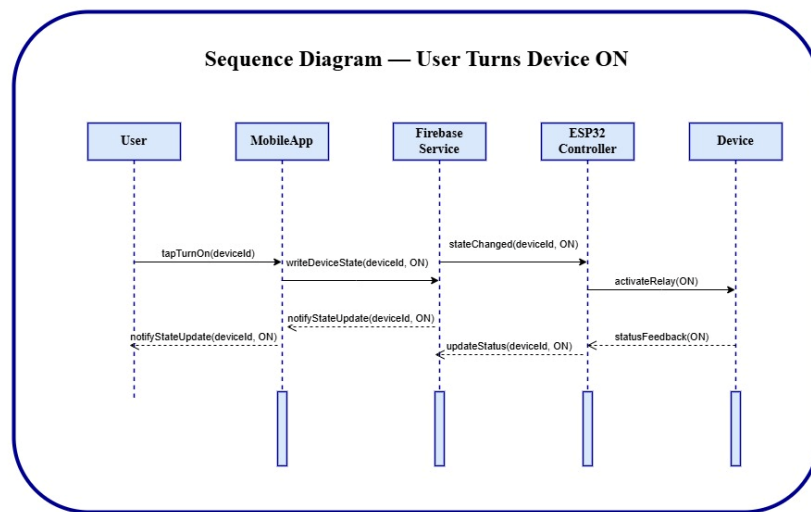


Figure 18: Interaction Diagram - User turns on Device

- **Interaction Diagram (Device Control Interaction)**

Figure 19 illustrates how a user command flows through the system when controlling a device. The Mobile App sends a control request to **FirestoreService**, which updates the device state in the **Firestore Realtime Database**. The **ESP32Controller** listens for changes, executes the command on the physical device through the relay module, and sends updated status back to Firestore, which the Mobile App then displays.

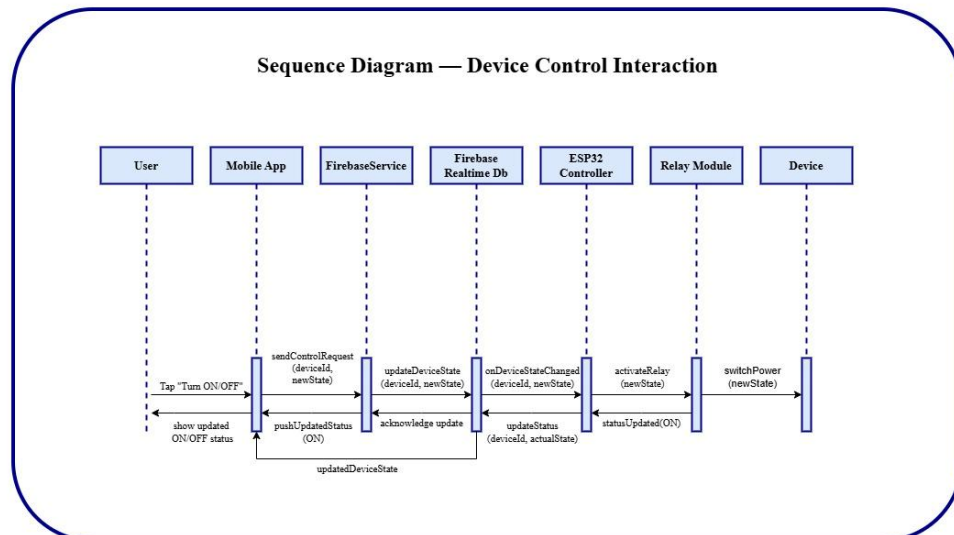


Figure 19: Interaction Diagram - Device Control Interaction

- **Interaction Diagram (Login Process)**

Figure 20 shows the authentication flow. The user enters credentials in the Mobile App, which sends a login request to **Firebase Authentication**. Firebase validates the credentials and returns the result. If successful, the Mobile App loads the user dashboard.

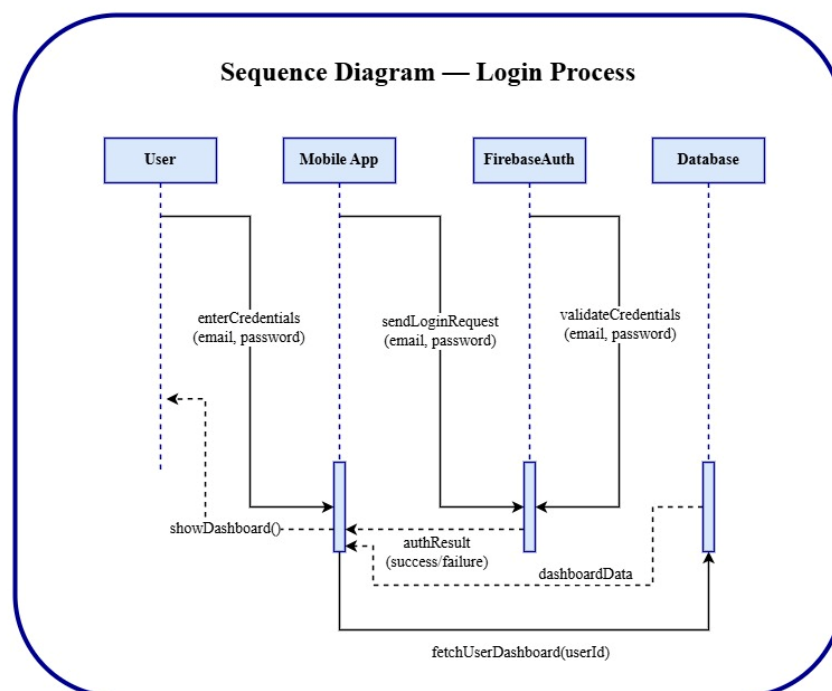


Figure 20: Interaction Diagram - Login Process

- **Automation Rule Execution**

Figure 21 shows how the system automatically reacts to sensor readings. When a sensor reports a new value, the Sensor Interface sends it to the ESP32Controller. The controller forwards the data to the AutomationManager, which checks the active automation rules. If a rule is triggered, the AutomationManager instructs the ESP32Controller to perform the required action, such as turning a device ON or OFF. The ESP32Controller activates the relay, updates the device state in Firebase, and the Mobile App receives the updated status in real time.

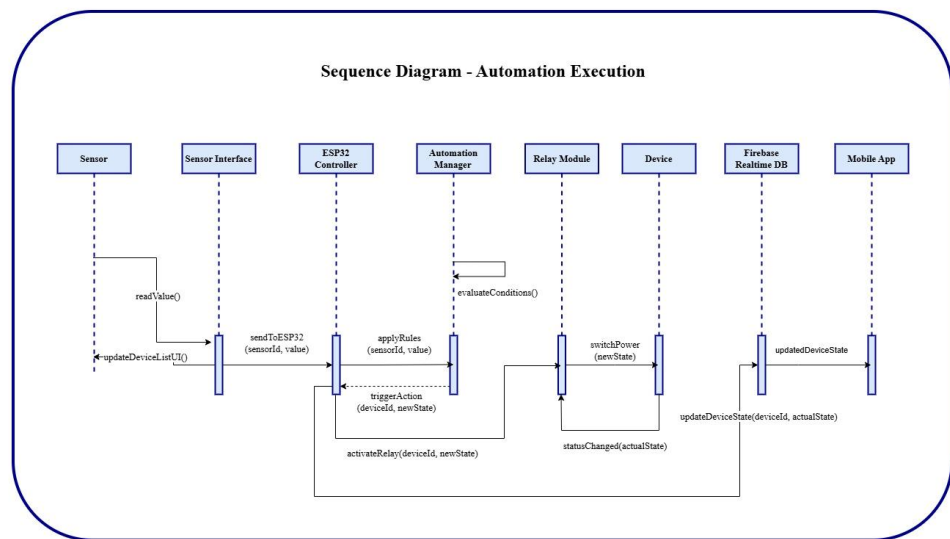


Figure 21: Interaction Diagram – Automation Rule Execution

4.4.3 State Transition Diagram

The state transition diagram describes how a system component changes its internal state in response to specific events. It is particularly relevant for automation features such as temperature-based fan control, motion-triggered lighting, or peak-time water management. The diagram models the lifecycle of an automated device, showing how it transitions between states like OFF, ON, AUTO, and ALERT based on sensor inputs and automation logic. **Figure 22** shows the state transition diagram for automated device control.

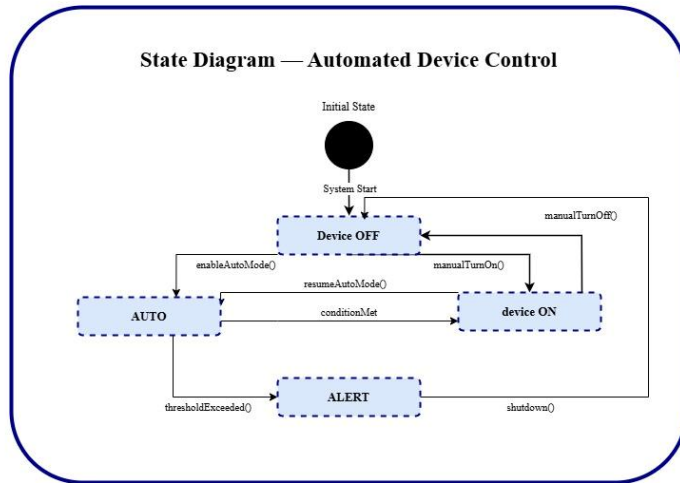


Figure 22: State Transition Diagram

4.5 Architectural Design

The proposed system follows a **three-tier client-server architecture**, ensuring clear separation of concerns, scalability, and maintainability. The **client tier** consists of the mobile application, which provides the user interface for device monitoring, manual control, and automation configuration. The **service tier** is implemented using Firebase services, which handle user authentication, real-time data synchronization, cloud storage, and communication between the mobile app and the controller. The **device tier** includes the ESP32-based controller, connected sensors, and relay modules that interact directly with the physical environment to execute automation rules and respond to user commands. The layered architecture and separation of concerns follow the recommended practices outlined in IEEE software design standards, ensuring maintainability and scalability (Society, 1998). **Figure 23** shows the smart home automation system architecture.

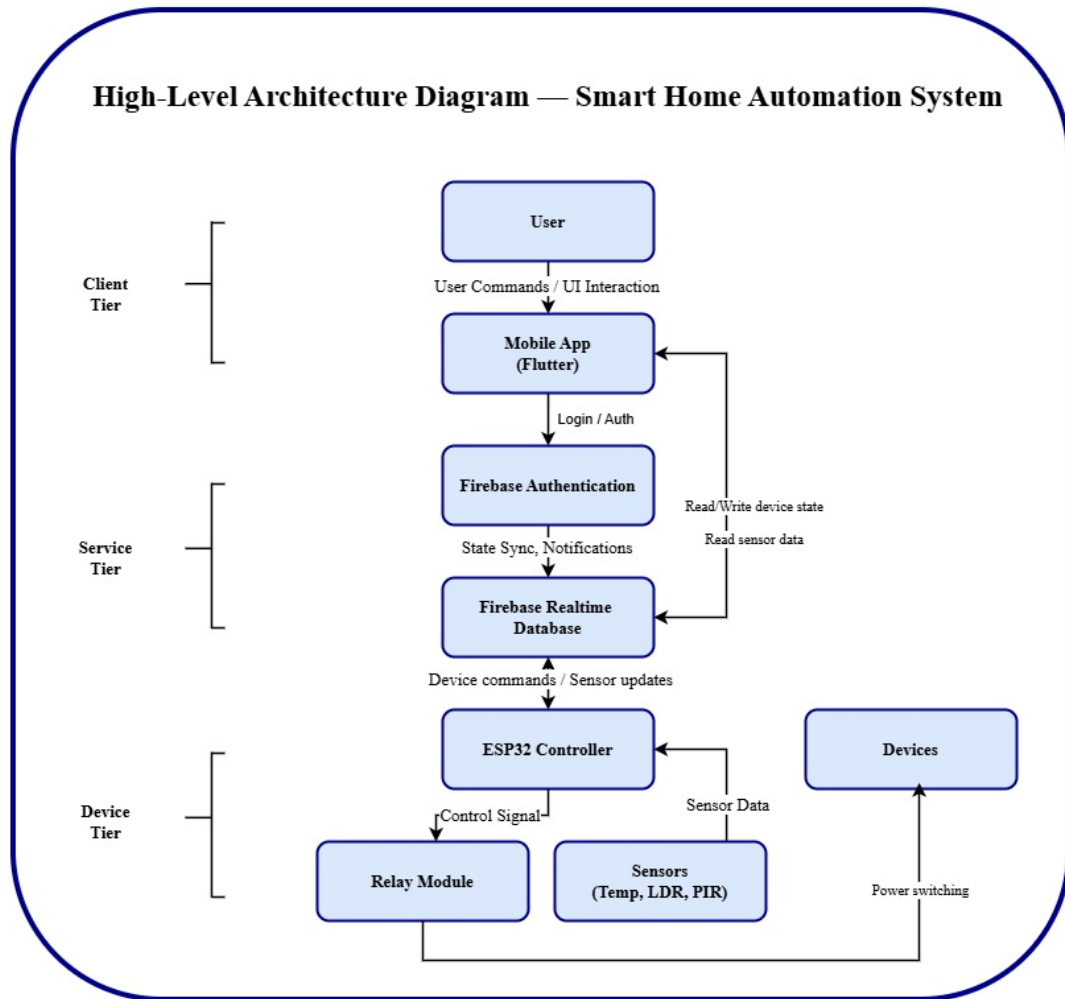


Figure 23: Architecture Diagram

4.5.1 UML Component diagram

The UML Component Diagram illustrates the modular structure of the smart home automation system. It includes components such as the **Mobile App**, which interacts with **Firebase Authentication** and **Firebase Realtime Database** for user login, device control, and data synchronization. At the device layer, the **ESP32 Controller** communicates with the **Sensor Interface** and **Relay Control** components to process sensor data and execute automation logic. **Figure 24** shows the UML Component diagram.

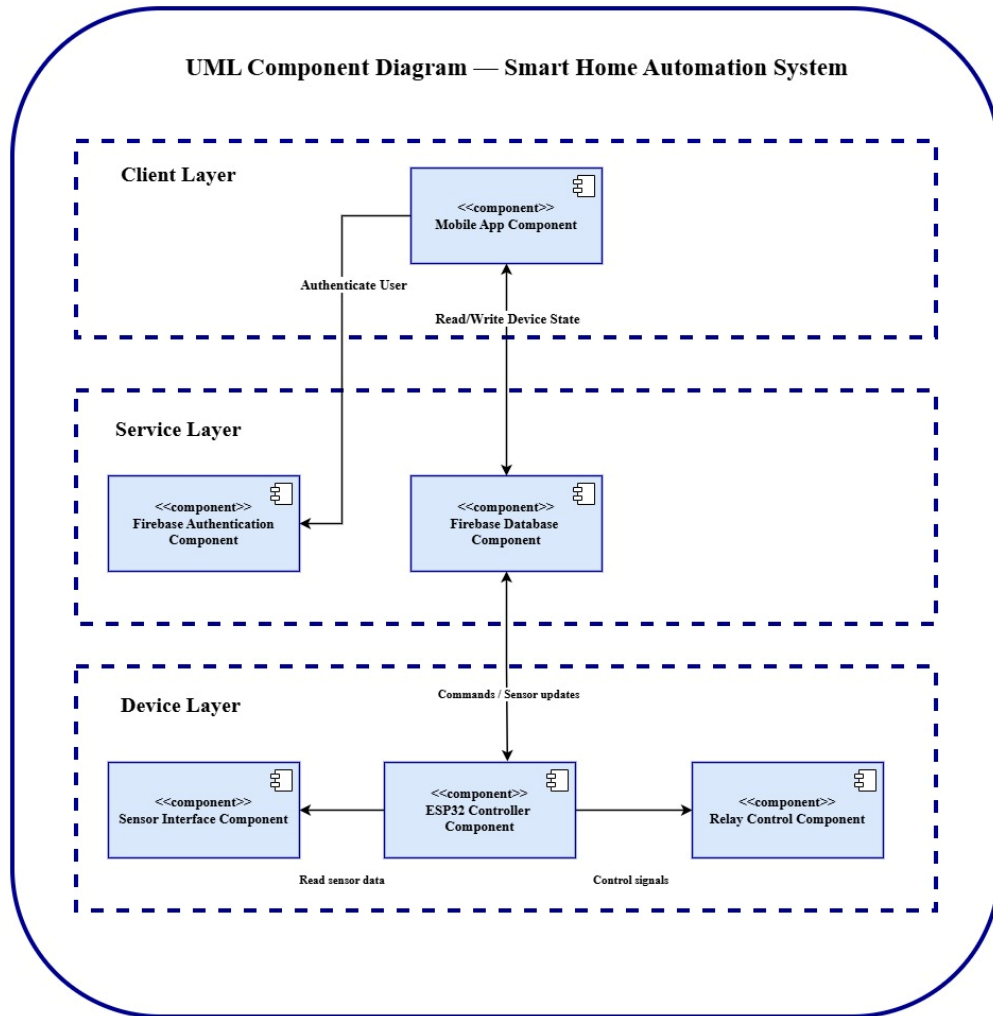


Figure 24: Component Diagram

4.6 Data Design

The system's data is organized around the main entities shown in the class diagram: User, Device, Sensor, Subscription, Automation settings, and ESP32 controller data. All information is stored in Firebase using a structured **JSON format**, which keeps the data lightweight and easy to sync across the mobile app and ESP32.

User accounts are stored through **Firebase Authentication**, while additional details such as subscription type and linked devices are kept in the **database**. Each device entry contains its ID, type, name, current state, and any threshold values. Devices may include one or more sensors, and each sensor stores its latest reading, type, and identifier.

The ESP32 updates sensor values and device states in Firebase, and the AutomationManager reads these values to apply automation rules. Subscription data is stored as a simple field under each user and is checked whenever a feature requires access control.

Overall, the data design ensures that all entities are stored in a clear, hierarchical structure, making it easy for the app, ESP32, and automation logic to read, update, and synchronize information in real time.

4.6.1 Data Dictionary

This section provides an alphabetical list of all major system entities, attributes, and methods used in the IoT-Based Smart Home Automation System. Each term includes its data type and a brief description of its purpose within the system. The Object-Oriented (OO) approach is followed, where objects, their attributes, and their methods are documented clearly. presents all system terms in alphabetical order. Each entry includes its data type and a concise description of its role within the system.

Table 37: Data Dictionary (OO Approach)

Terminology	Description (Data Type + Purpose)
a	
applyLightLevelLogic()	Method; applies curtain automation based on ambient light level.
applyMotionLogic()	Method; applies motion-based lighting rules.
applyPeakTimeLogic()	Method; triggers buzzer alerts during peak-time intervals.
applyTemperatureLogic()	Method; controls fan based on temperature thresholds.
applyWaterLevelLogic()	Method; controls water pump based on tank water level.
authReference: Map	Firestore authentication reference used for login and registration.
autoModeEnabled: Boolean	Indicates whether Automatic Mode is enabled.
c	
checkAccess()	Method; verifies if the user's subscription allows access to a feature.

d	
dbReference: Map	Firestore database reference for device and sensor data.
deviceId: String	Unique identifier assigned to each device.
deviceName: String	Human-readable name assigned to the device.
deviceType: String	Type of device (fan, light, curtain, pump).
e	
email: String	Registered email address of the user.
espId: String	Unique identifier of the ESP32 controller.
executeCommand()	Method; executes ON/OFF or automation commands on ESP32.
l	
login()	Method; authenticates a user using Firebase.
r	
readDeviceState()	Method; retrieves the current device state from Firebase.
readSensorData()	Method; reads data from connected sensors.
readValue()	Method; reads the latest value from a sensor.
register()	Method; registers a new user account.
runAutomationLogic()	Method; triggers all automation rules on the ESP32.

s	
sendToESP32()	Method; sends sensor data to the ESP32 controller.
sensorId: String	Unique identifier for each sensor.
sensorType: String	Type of sensor (temperature, motion, light, water).
state: Boolean	ON/OFF status of a device.
subscriptionType: String	User's subscription level.
syncSensorData()	Method; updates sensor readings in Firebase.
t	
threshold: int	Automation threshold value (temperature, light level, etc.).
turnOff()	Method; switches a device OFF.
turnOn()	Method; switches a device ON.
u	
updateFirebase()	Method; sends updated device/sensor data to Firebase.
updateStatus()	Method; updates the device's state in the system.
upgradePlan()	Method; updates the user's subscription tier.
upgradeSubscription()	Method; changes subscription through the mobile app.
userId: String	Unique identifier for each user.

v	
value: Float	Latest reading from a sensor.
w	
wifiStatus: Boolean	Indicates whether ESP32 is connected to Wi-Fi.
writeDeviceState()	Method; writes device state changes to Firebase.

4.7 User Interface Design

The user interface of the Smart Home Automation System is designed to be simple, intuitive, and responsive, allowing users to control devices and monitor sensor data with ease. The mobile app follows Material Design guidelines to ensure a clean layout, clear visual hierarchy, and consistent interaction patterns. Users can securely log in, view all connected devices, check their current status, and control them with a single tap. Real-time sensor readings—such as temperature, humidity, and light levels—are displayed clearly, and automation mode can be enabled or disabled at any time. The interface provides immediate visual feedback for every action through updated device states, refreshed sensor values, and helpful status indicators, ensuring a smooth and reliable user experience. The interface layout and visual hierarchy were designed in alignment with the official Material Design guidelines to ensure accessibility and consistency (Team, 2024).

4.7.1 Screen Images

Below are some of the screens showing our app display.

4.7.1.1 Splash Screen

This screen appears when the application launches. It displays the app's branding theme and transitions into the authentication flow. **Figure 25** shows the splash screen.

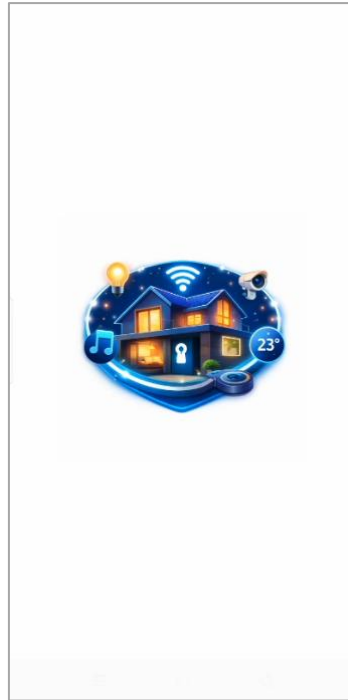


Figure 25: Splash Screen

4.7.1.2 Register Screen

This screen enables new users to create an account by providing their name, email, and password. It includes a clean form layout, a create-account button, and a link to return to the login screen. **Figure 26** shows the register screen.

Figure 26: Register Screen

4.7.1.3 Login Screen

This screen allows existing users to access the application by entering their email and password. It includes input validation, a login button, and a link to navigate to the registration screen. **Figure 27** shows the login screen.

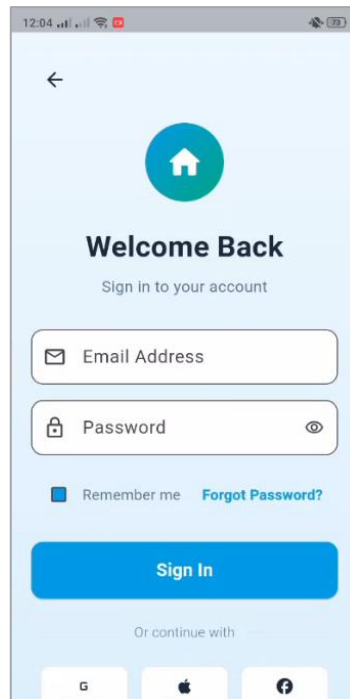


Figure 27: Login Screen

4.7.1.4 Dashboard Screen

The dashboard serves as the main navigation hub, allowing users to access devices, profile and overall system status. **Figure 28** shows dashboard screen.

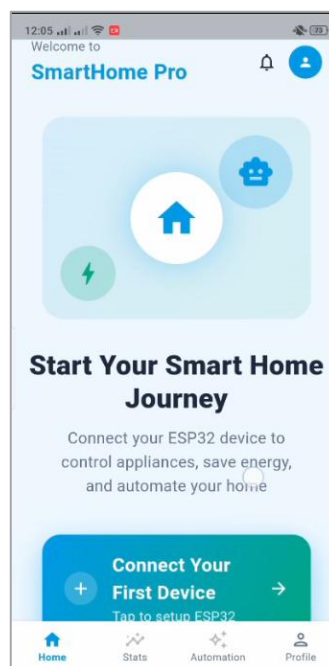


Figure 28: Dashboard Screen

4.7.2 Screen Objects and Actions

This section describes the key screen objects and the actions associated with them for two representative use cases.

- **Use Case 1: Turning a Device ON/OFF**

- **Screen Objects**

- **Device Card** – Displays device name, type, and current status
- **Toggle Button** – Allows switching the device ON or OFF
- **Status Indicator** – Shows real-time device state (e.g., green for ON, grey for OFF)
- **Feedback Message** – Confirms success or failure of the action

- **Actions**

1. The user taps the **Toggle Button** on a device card.
2. The system sends the command to update the device state.
3. The **Status Indicator** updates immediately.
4. A **feedback message** appears, such as:
 - “Device turned ON successfully”
 - “Unable to update device state”

- **Use Case 2: Viewing Device Status**

- **Screen Objects**

- **Device Status Card** – Shows device name, icon, and ON/OFF state
- **Sensor Value Display** – Shows any sensor values associated with that device (e.g., temperature for fan automation)
- **Refresh Icon** – Allows manual refresh of device status
- **History Button (optional)** – Opens past logs

- **Actions**

1. The user opens the Device Status Screen.
2. The system retrieves the latest device state from Firebase.
3. If the device is linked to a sensor, the sensor value is shown as part of the device status.
4. The user may tap Refresh to update the status.
5. The user may tap History to view past logs (if implemented).

4.8 Design Decisions

This section outlines the major design choices made during the development of the IoT-Based Smart Home Automation System. Each decision was taken to ensure that the system remains scalable, maintainable, and capable of handling real-time operations.

4.8.1 Selection of Object-Oriented Design

The system was designed using an Object-Oriented approach because the problem domain naturally consists of real-world entities such as Users, Devices, Sensors, and Controllers. Representing these components as classes allowed the system to maintain a clear modular structure. This approach supports encapsulation, reusability, and easier future expansion, especially when introducing new device types or automation rules. The OO model also simplifies maintenance by separating responsibilities across well-defined classes.

4.8.2 Choice of Firebase as Backend Platform

Firebase was selected as the backend due to its real-time synchronization capabilities, which are essential for updating device states and sensor readings without delay. Its built-in authentication system provides secure user management, while the cloud-hosted database eliminates the need for custom server deployment. The decision to use Firebase ensures low latency, high availability, and seamless communication between the mobile application and ESP32 devices.

4.8.3 Database Structure and Normalization Level

Since Firebase uses a NoSQL, document-based structure, traditional relational normalization rules do not apply. Instead, the database will be designed using a hierarchical JSON format that minimizes redundancy while keeping data easily accessible. The structure separates users, devices, and sensors into distinct paths, ensuring fast read and write operations. This design supports efficient real-time automation and reduces unnecessary data duplication.

4.8.4 Placement of Automation Logic on ESP32

Automation logic was intentionally executed on the ESP32 microcontroller rather than the mobile application or cloud. This decision ensures that automation features continue to function even when internet connectivity is unstable. The ESP32 can respond immediately to sensor changes, making it suitable for time-critical operations such as temperature control, motion detection, and water-level monitoring. This approach improves reliability and reduces dependency on network availability.

4.8.5 Communication and Data Exchange Approach

Communication between the mobile application, Firebase, and ESP32 devices uses lightweight JSON structures. This format is easy to parse and aligns naturally with Firebase's data model. The system uses a combination of event listeners and periodic updates to maintain synchronization across all components. This design

choice ensures efficient data exchange without overloading the network or microcontroller.

4.8.6 Separation of Subscription Management

Subscription management was implemented as a dedicated class to isolate access-control logic from user authentication. This separation improves maintainability and allows subscription-based features to be extended in the future without modifying core user functionality. It also ensures that subscription checks remain consistent across the system.

4.8.7 Abstraction of Devices and Sensors

Devices and sensors were modelled as separate classes to support flexibility in hardware integration. This abstraction allows new sensors or actuators to be added with minimal changes to the system's core logic. It also simplifies onboarding, automation processing, and communication with the ESP32 controller.

4.9 Summary

This chapter outlined how the system's architecture, data structures, and class design come together to support a reliable smart home automation solution. It explained how the system's requirements were translated into an object-oriented structure, how data flows between the mobile app, Firebase, and the ESP32, and why certain design choices such as using Firebase, placing automation logic on the microcontroller, and organizing the system into modular classes were made. These decisions ensure that the system remains responsive, scalable, and easy to maintain. Overall, the design presented in this chapter forms the foundation that enables the project to meet its core objectives of real-time control, automation, and seamless user interaction.

References

- Ahmad, S. K. (2023). Cloud-Based Smart Home Management System for Energy Optimization. *Springer Nature Computer Science*, 4(3), 90-101.
- Al-Fuqaha, A. G. (2015). A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376.
- Al-Fuqaha, A., Guibene, M., & Mohamed, N. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376.
- Al-Quraishi, M. M.-R. (2021). AI-Powered Smart Home Automation: Integrating IoT with Machine Learning for Energy Efficiency. *IEEE Access*, 9, 116234–116245.
- Arduino. (2024). *Arduino Official Documentation*. Retrieved November 10, 2024, from <https://docs.arduino.cc/>
- Association, I. S. (2025). *IEEE P2890—Standard for Smart Home Energy Efficiency and Interoperability*.
- Chen, M., Wang, L., & Zhang, H. (2025). Edge-AI for Real-Time IoT Automation: Reducing Latency in Smart Home Systems. *IEEE Internet of Things Journal*, 4500–4515.
- Developers, G. (2024, Novemberx 10). *Flutter Documentation*. Retrieved from <https://docs.flutter.dev/>
- Firebase, G. (2024). *Firebase Documentation*. Retrieved November 10, 2024, from <https://firebase.google.com/docs>
- Kaur, G. &. (2020). A Review on Smart Home Automation Techniques Using IoT. *Journal of Ambient Intelligence and Humanized Computing*, 11(8), 3465–3478.
- Khan, A. R., & Singh, S. P. (2025). Federated Learning for Privacy-Preserving Smart Home Energy Management. *ACM Transactions on Internet of Things*, 112–130.
- Nguyen, T., Patel, B., & Kumar, R. (2025). Low-Cost Arduino-Based Smart Plugs with Cloud-Integrated Energy Monitoring. *Proc. IEEE International Conference on Consumer Electronics (ICCE)*, (pp. 1–6). Las Vegas.
- Sharma, R. &. (2022). IoT-Based Smart Home Automation System Using Arduino and Mobile Application. *International Journal of Computer Applications*, 183(22), 15–20.
- Society, I. C. (1998). *IEEE Recommended Practice for Software Requirements Specifications*. New York.
- Sommerville, I. (2016). *Software Engineering*. Boston: Pearson Education.
- Sommerville, I. (2016). *Software Engineering, 10th Edition*. Harlow: Pearson Education.
- Systems, E. (2024). *ESP8266/ESP32 Technical Reference*. Retrieved from <https://www.espressif.com/en/support/documents/technical-documents>
- Team, G. M. (2024). *Material Design Guidelines*. Retrieved from <https://m3.material.io>
- TensorFlow. (2024). *TensorFlow Lite for Microcontrollers*. Retrieved November 10, 2024, from <https://www.tensorflow.org/lite/microcontrollers>